Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

AUTONOMOUS CYBER DEFENDER

Project Engineering

Year 4

David Jayakumar

G00419108

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Acknowledgements

I would like to thank my project supervisor Paul “Goat” Lennon for the guidance and support throughout this project. I would also like to thank the lecturers who taught the modules that underpinned this work for the last 4 years, and my classmates for the discussions, feedback, and encouragement along the way. Finally, to whoever picks up this project next I hope the handover section saves you some time, and Godspeed.

Table of Contents

1 Summary 8
2 Poster 9
3 Introduction 10
4 Background 11
4.1 Network Defence Concepts 11
4.2 Reinforcement Learning 11
4.3 OpenAI Gymnasium 12
4.4 Graph Neural Networks 12
4.5 CybORG Cage 4 12
4.6 Sim-to-Real Transfer 13
4.7 Containerlab 14
4.8 PyTorch & Implementation Tools 14
5 Project Architecture 15
6 Project Plan 17
7 System Implementation 19
7.1 Network Monitor 19
7.2 Intrusion Detector 19
7.3 Graph Builder 20
7.4 Agent Adapter 21
7.5 Action Executor 22
7.6 Red Agent 22
7.7 Main Loop 23
7.8 Frontend Dashboard 24
7.9 Evaluation & Comparison 24
8 Testing & Validation 24
8.1 Phase 1- Algorithm Validation 24
8.2 Phase 2 - Unit Tests 25
8.3 Integration Testing 25
8.4 CI/CD Pipeline 25
9 Phase 1: Algorithm Benchmarking Results 26
9.1 CartPole-v1 & FrozenLake-v1 - PPO 27

9.2 CartPole-v1 & FrozenLake-v1 - DQN 29
9.3 FrozenLake-v1 - A2C 30
9.4 Cross-Algorithm Comparison 31
10 Phase 2: CybORG Simulation Results 32
10.1 Cumulative Rewards 33
10.2 Per-Episode Performance 33
10.3 Smoothed Learning Curves 34
10.4 Action Distribution Analysis 35
11 Bridge Development and Sim-to-Real Transfer 35
11.1 Containerlab Network Topology 35
11.2 Baseline Deployment - No Action Masking 36
11.3 Masked Deployment - Inference-Time Action Masking 38
11.4 Action Distribution Comparison -Simulation vs Bridge 40
11.5 Sim-to-Real Gap Analysis 41
12 Problems & Solutions 42
12.1 Phase 1 - Algorithm Benchmarking 42
12.1.1 Laptop Overheating During Training 42
12.1.2 Jupyter Kernel Not Recognised on College Computer 42
12.1.3 FrozenLake Graph Bug - Reward Threshold Set to 475 42
12.1.4 PPO Timestep Count Off by 240 42
12.1.5 CI/CD Branch Push Logic Misunderstood 43
12.1.6 Is_slippery = True - Producing Poor FrozenLake Results 43
12.2 Phase 2 - CybORG Exploration 43
12.2.1 CybORG Cage 4 Import Errors - gym vs gymnasium 43
12.2.2 Training From Scratch - Consistently Degrading Reward 43
12.2.3 Initial Plans for Custom Network - Abandoning Mininet Plans 43
12.3 Bridge Implementation 44
12.3.1 Feature Dimension Mismatch 44
12.3.2 Agent Blind to Compromise - Feature 187 Never Set 44
12.3.3 Dead Graph Variable - Compromise Map Not Reaching the Agent 44
12.3.4 Container Name Resolution - CLAB_PREFIX Mismatch 44
12.3.5 Block Action - Index Out of Range when Container Already Disconnected 45
12.3.6 Stale Compromised Flags Between Test Runs 45

12.3.7 Stale Decoy Containers From Previous Runs 45
12.3.8 Iptables Not Available in Alpine Containers for Edge Actions 45
12.3.9 Edge Action Index Ordering Bug - Docker API vs Alphabetical 45
12.3.10 Stale .pyc Bytecode - Code Changes Not in Effect 46
12.3.11 /tmp Persists Across Docker Restarts in Alpine 46
12.3.12 DiscoverDeception Bug - Decoy Dict Not Shared 46
12.3.13 Get_last_action() Always Returning Sleep in evaluation.py 46
12.3.14 Infinite Restore Loop on Root-Compromised Hosts 47
12.3.15 Sim-to-Real Action Space Gap - Edge Actions as No-Ops 47
12.3.16 Feature Vector Sparsity in Deployment 47
13 Design Choices 49
13.1 RL Environment Selection - CybORG Cage 4 vs Alternatives 49
13.2 Network Simulation Tool - Mininet vs Containerlab 49
13.3 Bridge Topology - Full CAGE4 Replication vs Simplified Subset 49
13.4 Agent Strategy - Train from Scratch vs Inference with Pre-Trained Weights 50
13.5 Frontend Architecture - FastAPI vs stdlib http.server 50
13.6 Compromise Representation - File Markers vs On-Memory State 51
13.7 Which CybORG Agents to Use for Evaluation 51
13.8 Inference-Time Action Masking - Deployment vs Retraining 51
14 Ethics 52
15 Conclusion 53
16 Handover 54
16.1 Key Files 54
16.2 CI/CD 55
16.3 Known Limitations 55
16.4 Known Gotchas 55
16.5 Future Development 55
17 Appendix 57
17.1 Repository: Phase 1 57
17.2 Repository: Phase 2 57
17.3 Cage4-topology.yaml 57
17.4 Inspect_weights.py (view 192 input dimension) 65
17.5 Inspect_weights.py - Raw Output (192 feature dimension) 67

18 AI Usage 71
19 References 72

1 Summary

The aim of this project was to find out whether a Reinforcement Learning (RL) agent trained entirely in a network simulation could autonomously defend a real containerised network. This gets at a core question in cyber security: does a trained policy transfer hold when the environment changes from simulation to live infrastructure?

The work was split into two phases. Phase 1 benchmarked three RL algorithms. PPO, DQN, and A2C were all investigated on CartPole-v1 and FrozenLake-v1 using Stable-Baselines3[6]. There were four training runs per algorithm, learning curve analysis, and cross-algorithm evaluation. All three cracked both environments; PPO converged most reliably and was picked for Phase 2.

Phase 2 built the autonomous network defence system. A GNN-PPO agent was trained using PyTorch and PyTorch Geometric [8] on the CybORG CAGE4 enterprise network simulation [1]. This modelled the network as a graph of hosts and connections. A bridge layer was then built to deploy this trained policy against a real Containerlab network [9] running in Docker. This comprised of five components: a network monitor, a graph builder encoding the live container state into a 192-dimensional feature vector per node, an intrusion detector using sentinel files to track compromises, a red agent re-implementing the CybORG FiniteStateRedAgent FSM [17] via Docker exec calls, and an action executor translating policy decisions into real container operations. A live D3.js dashboard visualises the episode in real time. The system was managed across 15 two-week sprints using Agile Scrum, with automated CI/CD via GitHub Actions.

Evaluation confirmed that the policy transfer holds under inference-time action masking. In the unmasked bridge deployment, edge actions (AllowTrafficZone and BlockTrafficZone) accounted for 69% of all selections versus 51.5% in simulation, none of which had a real beneficial effect on the live network. With masking applied, Monitor transferred consistently (41% bridge vs 34% simulation). With the edge probability mass redistributing to active executable actions. This led to Analyse rising to 39% as the agent probed more under sparse observations, Remove to 10%, and DeployDecoy to 9%. Two key sim-to-real gaps were identified and addressed through inference-time action masking: unsupported firewall actions and feature vector sparsity [12], adding a concrete deployment case study to the sim-to-real transfer literature.

2 Poster

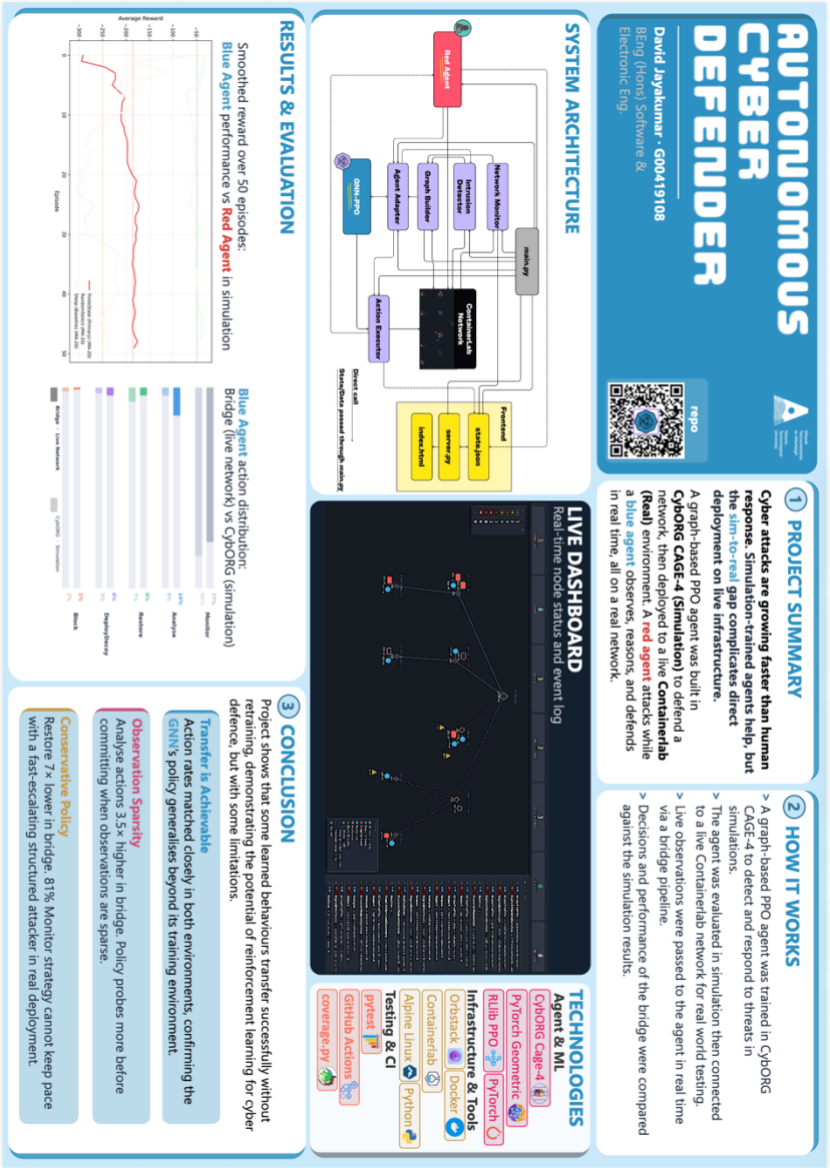

Figure 2-1 Poster of Project

3 Introduction

Cybersecurity threats against network systems are increasing in both frequency and complexity. Traditional rule-based defence systems rely on patterns. They also require constant manual updating by human security teams. As networks grow in scale and attackers adapt their techniques, purely human-driven defence becomes a bottleneck. There's a strong case for systems that can learn to detect and respond to threats autonomously.

Reinforcement Learning offers a solution: instead of being programmed with explicit rules, an RL agent learns a defensive policy. By interacting with an environment and receiving reward signals, its actions contain or fail to contain an attacker. When combined with Graph Neural Networks, which can natively represent the topology of a network, an agent can reason about network structure and connectivity a scalable way [7]. The key question though, is whether a policy trained entirely in simulation generalises when deployed against real infrastructure. This is the sim-to-real transfer problem this project set out to tackle.

The main goal was to build and evaluate an autonomous network defender. This is to determine whether a GNN-PPO policy trained in simulation retains effectiveness when deployed against a real containerised network. The work covered two phases: first, a deep study of three RL algorithms (PPO, DQN, and A2C) in benchmark environments to understand RL foundation; second, the design, implementation, and evaluation of a GNN-PPO-based defender operating in the CybORG CAGE 4 enterprise network simulation [1] and a real containerised network deployed using Containerlab [9].

The project ran across a full academic year. Managed using Agile Scrum with Jira for sprint planning and GitHub for version control and CI/CD.

4 Background

4.1 Network Defence Concepts

Network defence involves detecting and responding to malicious activity on a network. In enterprise environments, networks are typically divided into subnets with different trust levels. A blue team is responsible for defending the network. A red team (or attacker) attempts to compromise it. Common defensive actions available to a blue agent include: Monitor (polling hosts for suspicious processes or connections), Analyse (deep inspection of a host for signs of compromise), DeployDecoy (placing a honeypot service to trap and expose attackers), Remove (eliminating an attacker's user-level / root-level shell from a compromised host), and Restore (rebuilding a host to a clean state) [1].

Understanding the attacker's perspective is equally important. The typical attack sequence in CybORG follows privilege escalation: an attacker first performs network discovery (AggressiveServiceDiscovery to map the subnet), then exploits a discovered service (ExploitRemoteService to gain a user-level shell), and finally escalates privileges (PrivilegeEscalate to gain root access). A good blue agent needs to detect and interrupt this chain before root access is established.

4.2 Reinforcement Learning

Reinforcement Learning is a branch of machine learning. In it an agent learns to make logical decisions by interacting with an environment [2][3][4]. At each step, the agent observes the environment state, selects an action according to its current policy, and receives a scalar reward signal. The goal is to learn a policy that maximises the expected reward over time. The core loop is observing state -> select action -> environment steps -> receive reward -> observe new state.

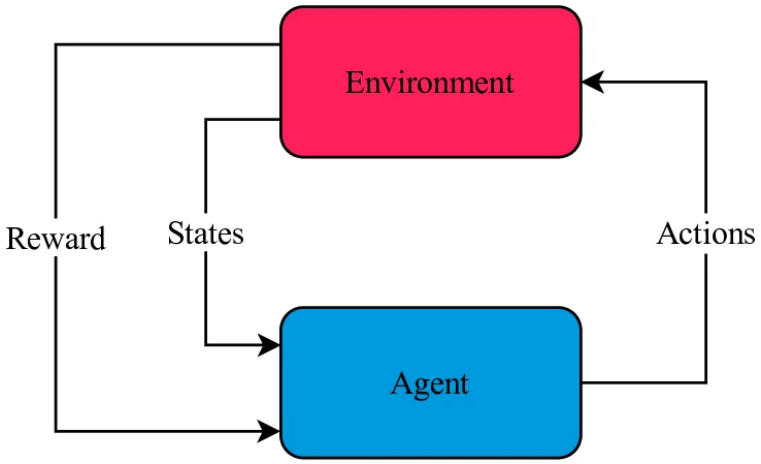

Figure 4-1 Reinforcement Learning core Loop Visualized

Three algorithms were tested and compared. Proximal Policy Optimisation (PPO) [2] is an on-policy actor-critic algorithm that updates its policy using a clipped objective. This prevents destructively large updates. This clipping mechanism is what gives PPO its smooth, stable learning

curves. Deep Q-Network (DQN) [3] is a value-based off-policy algorithm using a neural network to approximate Q-values. It joins this neural network with a replay buffer and a target network to stabilise training. DQN's off-policy nature means it can learn from past experiences (the replay buffer). Leads to its characteristic slow start followed by rapid improvement once the buffer is populated. Advantage Actor-Critic (A2C) [4] is an on-policy algorithm using both an actor (policy) and a critic (value estimate), with parallel environment rollouts. A2C typically converges fastest but can exhibit occasional large policy drops.

4.3 OpenAI Gymnasium

Gymnasium (formerly OpenAI Gym) [5] is a Python toolkit providing standard benchmark environments for RL research. This includes environments like CartPole, FrozenLake, or LunarLander. It defines a consistent API. env.reset() returns an initial observation. env.step(action), which advances the environment by one step and returns the observation, reward, terminated, truncated, info. Gymnasium provides the environments for testing RL algorithms but not the algorithm implementations themselves. Stable-Baselines3 [6] was used to provide those.

Two environments were used in Phase 1. CartPole-v1 is a classic control problem where a pole must be balanced on a moving cart by applying left or right forces. The observation space is a 4-element vector (cart position, cart velocity, pole angle, pole angular velocity). The action space is discrete (2 actions - left or right), and reward is +1 per timestep the pole remains upright. Maximum episode reward is 500. FrozenLake-v1 is a grid navigation task where an agent must cross a frozen lake from start (S) to goal (G) without falling into holes (H). The observation is a single integer (0-15) representing grid position. Reward is sparse: 1.0 for reaching the goal, 0 otherwise. The is_slippery=False variant was used, making moves deterministic. FrozenLake was chosen as it on a high-level represents a grid like a network topology.

4.4 Graph Neural Networks

Graph Neural Networks [7] are deep learning models designed for graph-structured data. A GNN operates via iterative message passing: each node aggregates feature information from its neighbours. This producing updated representations that encode both local node properties and its neighbourhood context. After several rounds of aggregation, a global pooling operation (e.g. mean pooling) produces a fixed-size graph-level embedding regardless of graph size.

Networks are naturally graph-structured: hosts are nodes and connections are edges. Representing the observation as a graph rather than a flat feature vector lets the agent reason about topology in a way that is both scalable and structure aware. This allows an agent to capture which hosts are connected, which are suspicious, and how compromise spreads. PyTorch Geometric [8] [23] was used to implement the GNN agent in this project, providing efficient message-passing primitives.

4.5 CybORG Cage 4

CybORG (Cyber Operations Research Gym) is an open-source RL environment developed by the TTCP (The Technical Cooperation Program) for research into autonomous cyber operations [1] [17] [18]. It simulates a multi-agent enterprise network scenario with red (attacker), blue

(defender), and green (regular user) agents. CybORG Cage 4 was released in February 2024 and is the current official challenge version.

Cage 4 uses an EnterpriseScenarioGenerator to produce a realistic multi-subnet enterprise topology. The network consists of multiple zones (restricted, operational, public access, contractor), each managed by a dedicated blue agent. CybORG's native observation is a flat host-state vector per agent. This project replaces it with a graph-structured observation. Each of the network hosts is encoded as a 192-dimensional feature vector, capturing node type, process state, compromise level, subnet membership, and decoy status. The red agent follows the FiniteStateRedAgent FSM from CybORG CAGE 4. It is a structured 9-action attack sequence with probabilistic state transitions covering discovery, exploitation, and privilege escalation. The environment's reward structure penalises the blue team for each compromised host and for the cost of defensive actions. A reward of 0 would represent perfect defence at zero cost, which is unachievable in practice.
Earlier versions of CybORG were considered: Cage 2 (April 2022) and Cage 3 (September 2022). CybORG++ (October 2024) [10] was also considered but not selected as it isn't an official TTCP release. Cage 4 was chosen for its more realistic enterprise scenario, larger and richer action space, and active maintenance by the TTCP team.

4.6 Sim-to-Real Transfer

A core challenge in deploying trained RL agents is the sim-to-real gap. A policy learned entirely in simulation may not transfer effectively to a real environment [11]. This gap arises because simulators necessarily abstract and idealise the systems they model. Observation spaces may encode richer metadata than what is measurable, and actions that are valid in simulation may be unavailable or behave differently on real transfers.
In network defence, this gap arises in two ways. First, action feasibility: CybORG models subnet firewall rules that don't exist in lightweight Containerlab deployments. Traffic-blocking actions (indices 64-79 of the CAGE 4 action space) are valid in simulation but have no real equivalent when containers lack iptables support. Second, observation sparsity: CybORG tracks rich simulation metadata per host (OS version, service pack, file system state) that can't be read from live containers, leaving a substantial portion of the 192-dimensional feature vector as zero at deployment.
A well-known fix for the action feasibility problem is inference-time action masking. Trained weights are left unchanged, but the output probability distribution is intercepted at deployment. Invalid action probabilities are zeroed out, and the remaining distribution is renormalised before sampling [12], [13]. Huang and Ontañón [12] prove that this procedure yields valid policy gradient updates, and Wilson et al. [13] demonstrate its application in OT cyber security RL specifically. Additional supporting work validates inference-time masking for temporally constrained action spaces [14], real-time strategy game deployments [15], and safety-critical constrained environments [16]. This approach was used in this project to prevent the agent selecting firewall actions unsupported by the live Containerlab environment.

4.7 Containerlab

Containerlab [9] is an open-source tool for deploying container-based network topologies using a YAML definition format. Unlike Mininet, which emulates networks using Linux kernel namespaces, Containerlab manages real Docker containers. Each node in the topology is a fully functional container with its own network interfaces, IP addressing, and routing table. This distinction mattered here: because the bridge layer interacts with the network exclusively through the Docker SDK. Containerlab-managed containers are accessible to the bridge components without any additional abstraction layer.

The CAGE 4 bridge topology was defined as a Containerlab YAML file specifying nine subnets mirroring CybORG's zone structure, with dedicated router, server, and user containers per zone. Containerlab handles container lifecycle management, including deploying, connecting, and destroying containers. The bridge components interact with the running containers directly via Docker exec calls. OrbStack [25] [26] was used on macOS to provide the Linux environment required to run Containerlab. While Containerlab provides realistic container networking, all nodes run on a single host machine, meaning traffic between containers does not traverse real physical infrastructure. This is a distinction relevant to deployment on live enterprise networks which was not accessible for this project.

4.8 PyTorch & Implementation Tools

Stable-Baselines3 [6] was used for PPO, DQN, and A2C implementations in Phase 1. PyTorch and PyTorch Geometric [8] were used for GNN graph building in Phase 2. The Docker SDK for Python [20] was used to execute actions on live containers and monitor container state. Containerlab [9] was used to deploy and manage Docker-based network topologies. GitHub Actions provided CI/CD pipeline automation. Jira was used for Agile Scrum sprint management throughout the project.

5 Project Architecture

The system architecture consists of six components operating in a closed-loop cycle at every episode step. The flow is:

Red Agent attacks the live network -> **Network Monitor** collects updated container state -> **Intrusion Detector** scans for compromise markers -> **Graph Builder** converts state to a PyTorch Geometric graph -> **Agent Adapter** (GNN-PPO) predicts a defensive action -> **Action Executor** applies the action to the live network.

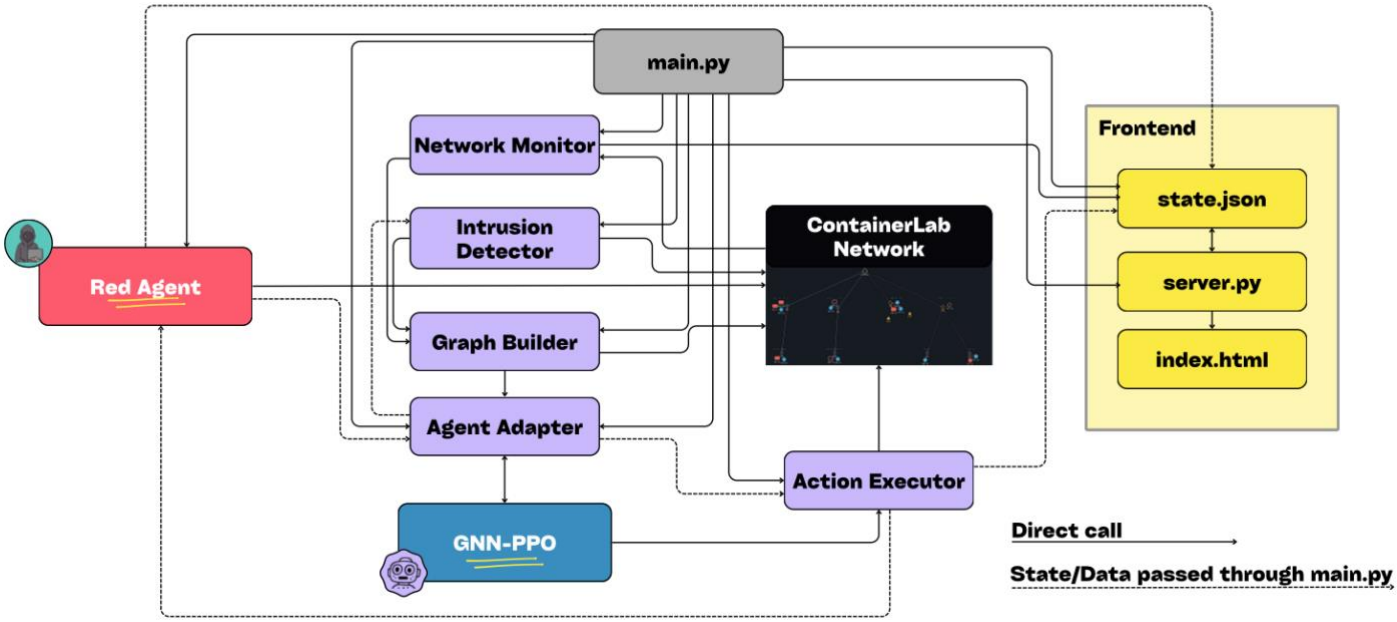

Figure 5-1 Architecture Diagram

The six components are:

Red Agent: Simulates the CybORG FiniteStateRedAgent FSM [1] [17] on live Docker containers. Maintains per-host FSM state (K/S/U/R and decoy-detected variants) and selects from 9 actions. Covering discovery, exploitation, privilege escalation, impact, and withdrawal, etc. all executed as Docker exec_run calls on Containerlab containers.

Network Monitor: Polls the Containerlab network at each step using the Docker SDK. It inspects running containers, classifying them by role (server/user/router), and collects process lists and management IP addresses.

Intrusion Detector: Scans each container for sentinel files placed by the red agent. The file /tmp/.compromised indicates user-level compromise (level 1), while /root/.compromised indicates root-level compromise (level 2). This is the bridge's Docker-native equivalent of CybORG's session tracking.

Graph Builder: Converts the network state into a PyTorch Geometric Data object [8]. Each of the hosts becomes a node with a 192-dimensional feature vector. Encodes system type, platform, process state, compromise level, subnet membership, and decoy flags. Edges connect hosts within the same subnet.

Agent Adapter (GNN-PPO): Receives the graph, passes it through the GNN encoder via message passing over edges, produces a graph-level embedding, and feeds this to the PPO policy to produce a discrete action integer (0-80). A global phase vector encodes the current episode phase as a one-hot input.
Action Executor: Translates the action integer into a concrete Docker operation: Analyse (ps aux), Remove (network disconnect), Restore (container restart and reconnect), or DeployDecoy (Containerlab YAML deploy of a nginx:alpine decoy). Edge actions (64-79) are optionally masked at inference time as they require iptables support unavailable in Containerlab containers.

The per-step sequence is shown in Figure 1-2.

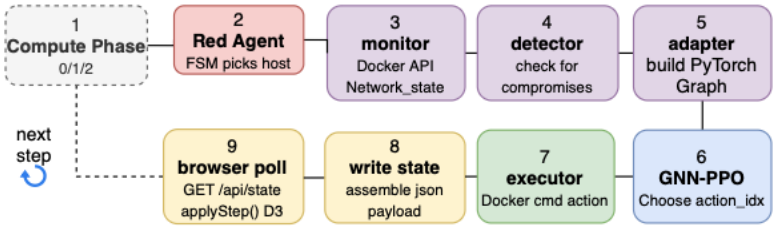

Figure 5-2 How each component fits in the sequence at each step

The episode runs for 100 steps divided into three phases: steps 0-33 (Recon), 34-66 (Escalation), and 67-99 (Impact). Mirrors CybORG's EnterpriseScenarioGenerator mission phase structure. The phase is passed to the agent at each step as a one-hot global vector, allowing the policy to adapt its behaviour as the episode progresses.

6 Project Plan

Agile Scrum was used to manage the project across both semesters. Jira used for backlog management, sprint planning, and progress tracking. GitHub Actions CI/CD pipeline was set up and maintained throughout, running automated tests on every push to the development branch. The initial project plan, produced at the start of Semester 1, is shown below:

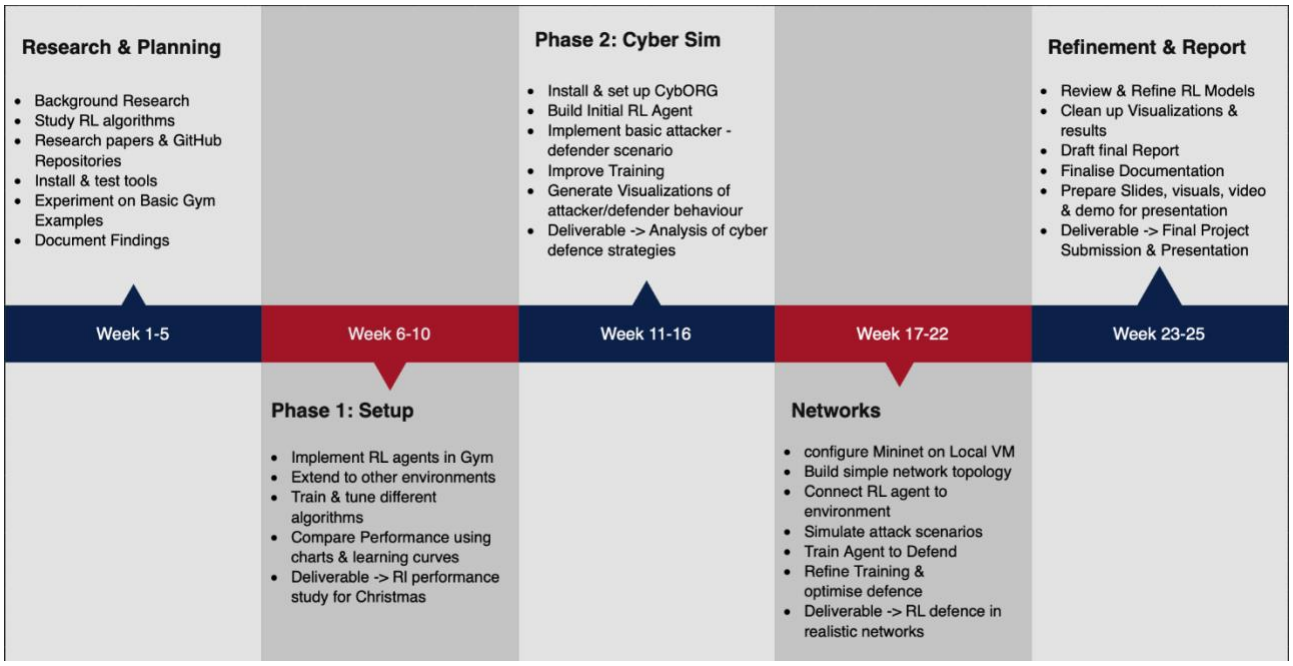

Figure 6-1 Initial Timeline Plan

The initial plan got the right structure: phase 1 of RL benchmarking followed by network deployment in phase 2, though several specifics changed during execution. The planned Mininet deployment was replaced with Containerlab after researching Docker-native network topologies. The bridge deployment phase expanded significantly once the complexity of connecting a trained simulation agent to live containers became clear, and evaluation work was pushed into the final sprints as a result.

Actual execution followed 15 two-week sprints across Semester 1, a Christmas break, and Semester 2:

Sprint(s)	Period	Goal
1-2	Sep-Oct 2025	RL foundations, Gymnasium research, PPO on CartPole
3-5	Oct-Nov 2025	DQN + A2C comparisons, algorithm benchmarking, CI/CD setup
6-7	Nov-Dec 2025	CybORG CAGE4 setup, blue agent action exploration

Break	Dec-Jan	CybORG deep-dive, blue and red agent behaviour study
8-9	Jan-Feb 2026	GNN-PPO training, CybORG evaluation, Containerlab setup
10-11	Feb-Mar 2026	Bridge components, intrusion detector, unit and integration tests
12-13	Mar-Apr 2026	Bridge evaluation, sim-to-real comparison, live dashboard, report

Overall project progress is shown in the Jira cumulative flow diagram below:

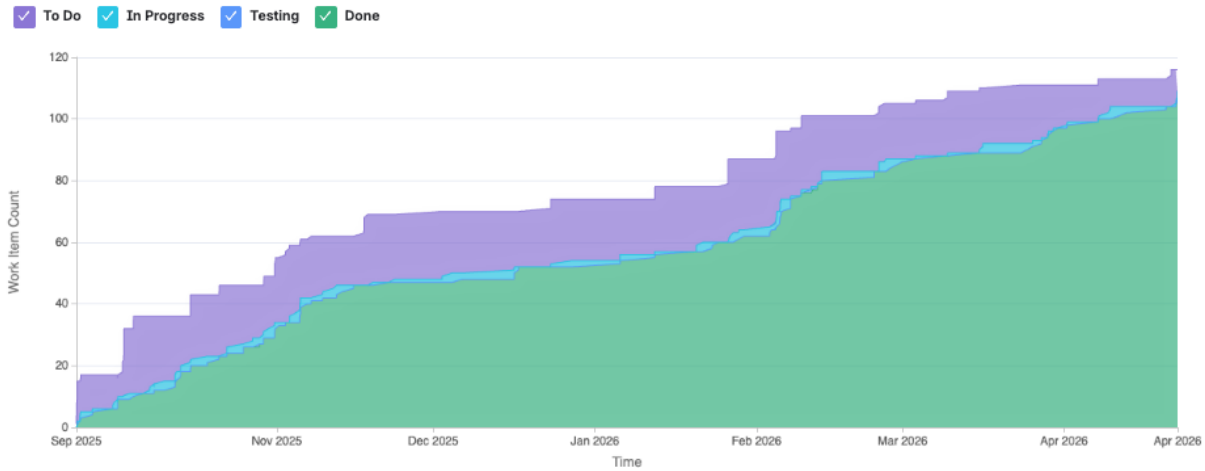

Figure 6-2 Jira Cumulative Flow Chart 09/25 - 04/26

The green (Done) band grows steadily from September 2025 through to April 2026. The gap between created and completed work items closing consistently shows work completion. By around 105 of 115, work items had been completed, with the remaining items corresponding to report and documentation tasks completed in the final week.

7 System Implementation

7.1 Network Monitor

The Network Monitor (network_monitor.py) polls the state of the live Containerlab network at every episode step using the Docker SDK for Python. It discovers all running containers whose names are prefixed with clab-. Classifying each host by role (server, user, or router) based on a predefined mapping of container name patterns. For each container it collects the container's management IP address and a process list retrieved via the Docker exec API [20].

A key design requirement is that containers are returned in a consistent slot-sorted order at every step. The CAGE 4 action space maps host index to container via a fixed slot (host_idx 0-5 = servers, 6-15 = users). The monitor sorts servers and users alphabetically by container name on every call. This so the same container always maps to the same host_idx at every step. This is needed for the graph builder and agent adapter to produce valid action targets.

7.2 Intrusion Detector

The Intrusion Detector (intrusion_detector.py) scans each container for the presence of sentinel files placed by the red agent. Two files are checked per container:

- /tmp/.compromised , present if the red agent has gained user-level shell access (compromise level 1)
- /root/.compromised , present if the red agent has escalated to root (compromise level 2)

```
def scan(self, containers):
    print("scanning for compromised markers on hosts")

    results = {}
    for c in containers:
        name = c["clean_name"]
        results[name] = self._check_container(name)
    return results
```

Figure 7-1 scan() Function

This approach was picked as the Docker-native equivalent of CybORG's session tracking. In CybORG, compromise is represented by session objects in the hosts state [1]. In the bridge, it is represented by attacker-placed sentinel files. The intrusion detector's output is fed directly into the graph builder to set the was_compromised feature (index 187) for each node.

One issue that came up during validation was that Restore only cleared /tmp/.compromised, leaving /root/.compromised intact after a container restart. The intrusion detector kept reading root-level compromise on the restarted host, causing the agent to select Restore on the same host indefinitely. The fix was to clear both markers in _restore():

```python
def _restore(self, full_name, clean_name):
    self._blocked_hosts.discard(clean_name)
    try:
        container = self.client.containers.get(full_name)
        container.reload()
        connected = list(container.attrs["NetworkSettings"]["Networks"].keys())
        mgmt_network = self._get_mgmt_network(container)
        was_blocked  = mgmt_network and mgmt_network not in connected
        container.restart()
        container.exec_run("rm -f /tmp/.compromised /root/.compromised")
        if was_blocked and mgmt_network:
            self.client.networks.get(mgmt_network).connect(container)
            print(f"[Executor] Reconnected {clean_name} to {mgmt_network}")
        print(f"[Executor] Restored {clean_name}")
        return {"action_type": "Restore", "target": clean_name, "result": "restarted"}
    except Exception as e:
        print(f"Error Restore failed {clean_name}: {e}")
        return {"action_type": "Restore", "target": clean_name, "result": f"error: {e}"}
```

Figure 7-2 restore() Function clearing flags

This followed CybORG's approach of a full system reboot no matter compromise level.

7.3 Graph Builder

The Graph Builder (graph_builder.py) converts the enriched network state dictionary into a PyTorch Geometric Data object [8] at every episode step. Each of the network hosts (servers, users) becomes a node encoded as a **192-dimensional feature vector**. The layout mirrors the trained agent's expected input exactly [19], as determined by inspecting the model weight shapes:

Indices	Feature
0	SystemNode flag (servers and users)
3	InternetNode flag (internet router only)
5	x64 architecture (always 1.0)
55-57	Node role one-hot: user / server / router
60	INIT process (/bin/sh present in all containers)
64	SSHD process (set if sshd in ps output)
70	APACHE2/nginx process (set if web server detected)
78	SHELL process (always 1.0)
99	Suspicious pid (set if compromise_level ≥ 1)
100	Is decoy (set if container is in executor._decoys)
178-186	Subnet one-hot (9 subnets, alphabetical order)
187	was_compromised (1.0 if compromise_level ≥ 1)
188	was_scanned (1.0 if red FSM state ≥ S)

Figure 7-3 Table defines indices of the feature vector used to characterise each node

The feature indices were determined by running inspect_weights.py (Appendix 1.3). It loads the trained weight file and maps each input dimension to its corresponding CybORG observation class. This made it clear which indices the bridge should populate, and which are simulation-only. Indices not listed, including OS version, service pack, and file system metadata (indices 6-53, 103-177), are left as zero because they encode CybORG simulation metadata with no real container equivalent. This is consistent with the trained agent's zero-default for unknown state and is covered further as a sim-to-real gap in Section 11.
Edges are added between hosts within the same subnet. The graph is constructed fresh each step, reflecting current live state.

7.4 Agent Adapter

The Agent Adapter (agent_adapter.py) loads the trained GNN-PPO weights [22] and runs a forward pass at each step to produce a discrete action integer. The action space covers **81 discrete actions** (indices 0-80): node actions (0-63), edge/firewall actions (64-79), and Monitor (80).
The episode is divided into three phases (Recon steps 0-33, Escalation 34-66, and Impact 67-99), which are passed to the agent as a one-hot global vector:

```
# mission phase default: phase 0 (pre-attack) = [1, 0, 0]
global_vec = torch.zeros(1, 3)
global_vec[0, phase] = 1.0
```

Figure 7-4 Global Phase calculation

Because edge actions 64-79 have no defensive effect in the Containerlab environment (while implemented using ip route blackhole, the red agent bypasses the network routing entirely), inference-time action masking is applied. The Categorical distribution is intercepted before sampling, invalid probabilities are zeroed, and the remaining distribution is renormalised:

```
# Zero edge action probs (64-79)
with torch.no_grad():
    distro = self.agent.actor(*state)
    probs = distro.probs.clone()
    probs[..., 64:80] = 0.0
    probs = probs / probs.sum(-1, keepdim=True)
    action = torch.distributions.Categorical(probs=probs).sample().item()
```

Figure 7-5 Inference time Action Masking

Argmax() was tried first instead of sample(), but always returned Monitor (80).This was because monitor was the single highest-probability action when the feature vector was sparse. Sampling preserves the stochastic behaviour the agent was trained with. **This masking approach does not modify the trained weights and is consistent with standard deployment practice [12], [13].**

7.5 Action Executor

The Action Executor (action_executor.py) translates the action integer into a Docker operation. The four executable node action types are:

Analyse (type 0): Runs container.exec_run("ps aux") on the target host. Does not modify container state.

Remove (type 1): Disconnects the target container from all non-management Docker networks via the Docker SDK [20], cutting the attacker's network access.

Restore (type 2): Restarts the container [21], reconnects it to its original networks, and clears both sentinel files (rm -f /tmp/.compromised /root/.compromised). This matches CybORG's full re-image semantics as described above (Section 7.2).

DeployDecoy (type 3): Generates a Containerlab YAML file [24] dynamically and deploys an nginx:alpine container as a new Containerlab-managed decoy host on the target's subnet.

```
topology_name = f"decoy_{clean_name}"
topology = {
    "name": topology_name,
    "mgmt": {"network": mgmt_network},
    "topology": {
        "nodes": {
            "decoy-host": {
                "kind": "linux",
                "image": "nginx:alpine",
            }
        }
    }
}
```

Figure 7-6 DecoyDeploy YAML generation

An issue arose during DeployDecoy development. The management network name had to be resolved dynamically, since hardcoding "clab-cage4-defense-network" caused Containerlab to fail. The fix was to inspect the container's existing networks via the Docker SDK, and use the network name beginning with "clab". Edge actions (64-79) are treated as no-ops at this layer since masking prevents them from being selected.

7.6 Red Agent

The Red Agent (red_agent.py) re-implements the CybORG FiniteStateRedAgent FSM [1] [17] on live Docker containers. Each container starts in state K (Known) and transitions through S (Scanned), U (User access), and R (Root access) based on action outcomes. The FSM transition matrices are copied directly from the CybORG source [18].

The red agent is a re-implementation rather than a wrapper around CybORG's red agent. This is because CybORG operates on a simulated environment object and can't execute actions on live Docker containers directly.

The 9 red actions are mapped to Docker exec_run calls:

Action	Docker Operation
DiscoverRemoteSystems	ping -c 1 <router_ip>
AggressiveServiceDiscovery	nc -zv <container_ip> 22 80 443
StealthServiceDiscovery	nc -zv <container_ip> 22 80
ExploitRemoteService	touch /tmp/.compromised
PrivilegeEscalate	touch /root/.compromised
Impact	dd if=/dev/zero of=/tmp/junk bs=1M count=10
DegradeServices	dd if=/dev/zero of=/tmp/degraded bs=1M count=20
Withdraw	rm -f /tmp/.compromised /root/.compromised /tmp/junk /tmp/degraded

Figure 7-7 Red Agent Docker Operation Mapping

Host selection follows a 75% server preference, sourced from FiniteStateRedAgent._choose_host() [18]. The red agent's per-host FSM states are also used to set the was_scanned feature (index 188) in the graph builder. A host is marked as scanned once the FSM state reaches S or higher.

7.7 Main Loop

The main episode loop (main.py) runs for 100 steps, wiring all six components in sequence at each step:

- Red Agent selects and executes an attack action on the live network
- Network Monitor polls all containers and collects current state
- Intrusion Detector scans containers for sentinel files
- Graph Builder constructs the observation graph with enriched features
- Agent Adapter runs a forward pass and returns a discrete action integer
- Action Executor applies the action to the target container

The per-step pipeline is shown below:

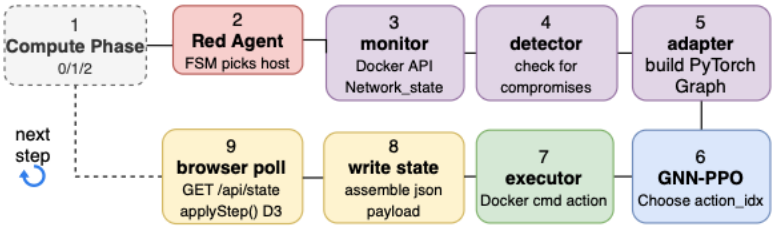

Figure 7-8 Components at each step in the loop

On startup, stale decoys and sentinel files from any previous run are cleared. The current state.json is written automatically after each step (via tmp file rename) for the dashboard server.

7.8 Frontend Dashboard

A live web dashboard (frontend/index.html, bridge/server.py) visualises the network state during each episode. A lightweight stdlib HTTP server serves two endpoints: GET / returns the dashboard HTML and GET /api/state returns the current state.json. The dashboard polls /api/state every 1.5 seconds.

The topology is rendered as a D3.js v7 force-directed graph with hull overlays per subnet. Node colours reflect live state: clean (blue), compromised (pulsing red), restored (green), decoy (yellow). The node and link arrays are hardcoded in JavaScript and mirror cage4-topology.yaml exactly, never changing at runtime. Only per-node state is dynamic. A live event log, FSM state chip overlay, and phase badge complete the interface.

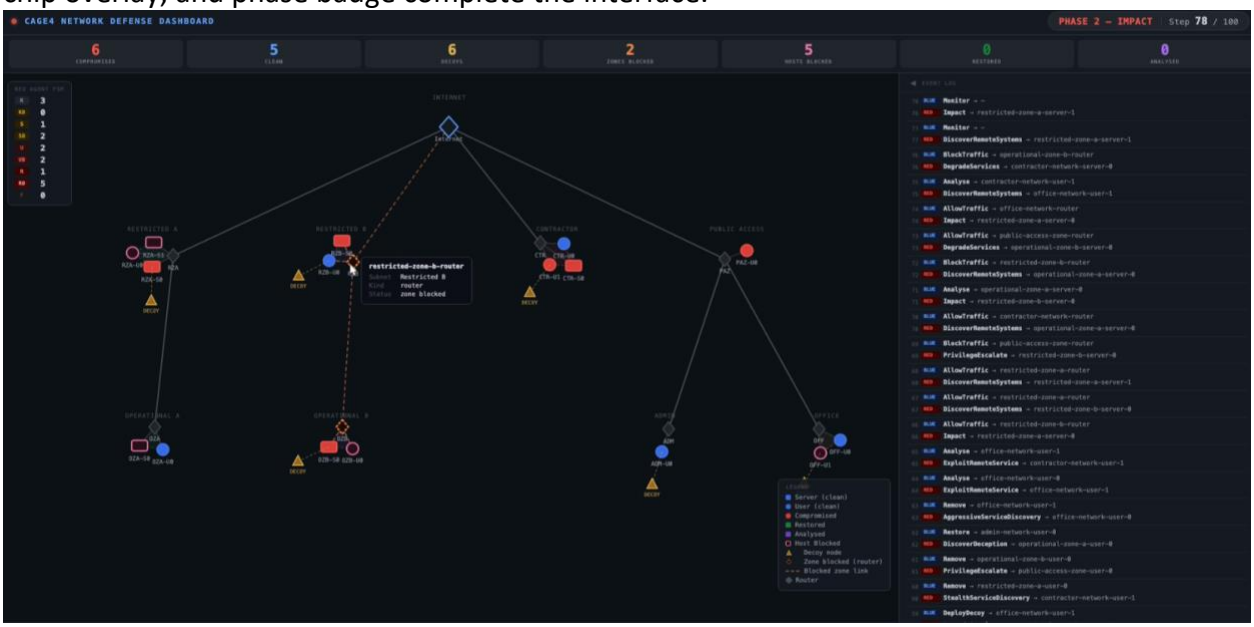

Figure 7-9 Live D3.js Dashboard

7.9 Evaluation & Comparison

A set of evaluation and analysis scripts supports the results presented in Section 11. evaluate_cyborg.py runs the trained agent inside the CybORG simulation and logs per-episode rewards and action distributions. evaluation.py runs a controlled 100-step episode on the live bridge network, logging every red and blue action to a timestamped CSV. An optional --mask flag enables inference-time action masking for comparative evaluation. compare.py loads the latest bridge and CybORG CSVs and produces the action distribution comparison. plot_bridge_analysis.py generates the compromise timeline, red/blue timeline, and action effectiveness plots from the bridge CSV. plot_comparison.py produces the side-by-side bar chart comparing bridge and CybORG action distributions used in Section 11.

8 Testing & Validation

8.1 Phase 1- Algorithm Validation

Phase 1 was built in a separate repo. Validation was performed through training and evaluation runs rather than a formal unit test suite. This was because the outputs being validated were

learning curves and policy performance rather than software components. Each algorithm (PPO, DQN, A2C) was run for four independent seeds to verify reproducibility. Evaluation curves were generated by loading saved checkpoints and running the policy deterministically (deterministic=True). This confirmed learned performance was stable and not an artefact of a single run based on a fluke. Results are documented in Section 9.

8.2 Phase 2 - Unit Tests

A pytest test suite was written for the bridge system covering all major components. 20 tests were written across 6 modules. Key tests included:

- **Action Executor:** test_analyse_action, test_block_and_restore, test_agent_pipeline , verifying actions execute without error and return expected outcomes on live containers.
- **Graph Builder:** test_graph_correct_shape, test_servers_and_users_classified_correctly, test_dimension_matches_weights , verifying node feature dimensions, edge indices, and node classifications.
- **Intrusion Detector:** test_user_compromise_detected, test_root_compromise_detected , verifying the detector correctly identifies user-level and root-level compromise via sentinel files on live containers.
- **Network Monitor:** test_containers_have_ip, test_processes_collected , verifying the Docker-based monitor collects valid state from running containers.
- **Red Agent:** test_initial_states_are_known, test_step_returns_valid_action, test_fsm_progresses_over_steps , verifying the FSM follows its attack sequence correctly.

20 tests pass and 1 is skipped in the final implementation. The skipped test (test_decoy_deploy) depends on Containerlab being installed, which is not available in the CI environment. However, it was validated manually against the OrbStack deployment locally.

8.3 Integration Testing

Integration testing verified the end-to-end bridge pipeline: network monitor -> intrusion detector -> graph builder -> agent adapter -> action executor. The full pipeline was run against the live Containerlab network to confirm all components interact correctly. Ensured observations flow without shape mismatches, and action execution completes without errors across all action types.

8.4 CI/CD Pipeline

A GitHub Actions CI/CD pipeline was set up and maintained throughout. The pipeline triggers on pushes to develop, feature/**, and SCRUM-**-feature/** branches, and on pull requests to develop and main. This follows a GitFlow methodology where features are developed in isolation and merged to develop before releases to main.

Steps: checkout code -> set up Python 3.11 -> spin up Docker Compose test topology -> install dependencies -> run pytest with coverage. The final CI run showed **20 passed, 1 skipped, 73% coverage**. Scrum-aware branch naming (SCRUM-**-feature/**) was added to enable automatic Jira ticket status transitions on push.

```
============= 20 passed, 1 skipped, 1 warning in 194.59s (0:03:14) =============
Name                                                                                                                    Stmts   Miss  Cover
-----------------------------------------------------------------------------------------------------------------------------------------------
/home/runner/work/Network_Defender_FYP/Network_Defender_FYP/trained-agent/models/cage4.py                                 247     81    67%
/home/runner/work/Network_Defender_FYP/Network_Defender_FYP/trained-agent/models/memory_buffer.py                          52     31    40%
/home/runner/work/Network_Defender_FYP/Network_Defender_FYP/trained-agent/models/utils.py                                  33     30     9%
action_executor.py                                                                                                        177     86    51%
agent_adapter.py                                                                                                           42      6    86%
graph_builder.py                                                                                                          106     14    87%
intrusion_detector.py                                                                                                      39      6    85%
network_monitor.py                                                                                                         79     13    84%
red_agent.py                                                                                                              105     22    79%
tests/conftest.py                                                                                                           4      0   100%
tests/test_action_executor.py                                                                                              45      8    82%
tests/test_agent_adapter.py                                                                                                23      4    83%
tests/test_graph_builder.py                                                                                                40      1    98%
tests/test_intrusion_detector.py                                                                                           49      0   100%
tests/test_monitor.py                                                                                                      23      0   100%
tests/test_pipeline.py                                                                                                     22      1    95%
tests/test_red_agent.py                                                                                                    31      0   100%
-----------------------------------------------------------------------------------------------------------------------------------------------
TOTAL                                                                                                                     1117    303    73%
```

Figure 8-1 GitHub Actions passing

9 Phase 1: Algorithm Benchmarking Results

All algorithms were trained using Stable-Baselines3 [6]. There were four independent runs per algorithm using different the same fixed seeds to verify reproducibility, plus one additional run at seed 23 for cross-seed comparison. PPO and DQN were evaluated on both CartPole-v1 and FrozenLake-v1; A2C was evaluated on FrozenLake-v1 only. CartPole-v1 runs used shorter training budgets (10,000 and 20,000 timesteps) to observe convergence behaviour at different training lengths. FrozenLake-v1 runs used 100,000 timesteps across all three algorithms. Evaluation curves were generated by pausing training every 200 timesteps and evaluating the policy deterministically over multiple episodes (deterministic=True ensures no exploration noise, reflecting true policy performance).

9.1 CartPole-v1 & FrozenLake-v1 - PPO

PPO was trained on CartPole-v1 for 10,000 and 20,000 timesteps. Training curves show three distinct phases. In episodes 0-100, rewards are consistently below 50 as PPO takes random actions to build experience. This is the exploration phase where the agent has not yet learned the relationship between observations and optimal actions. In episodes 100-200, occasional spikes to 100-150 appear as better policies emerge. Instability remains due to PPO's clipped gradient updates switching between exploration and exploitation. From episode 200 onwards, rewards climb sharply to 250-350+, marking policy convergence. PPO now reliably balances the pole for extended durations.

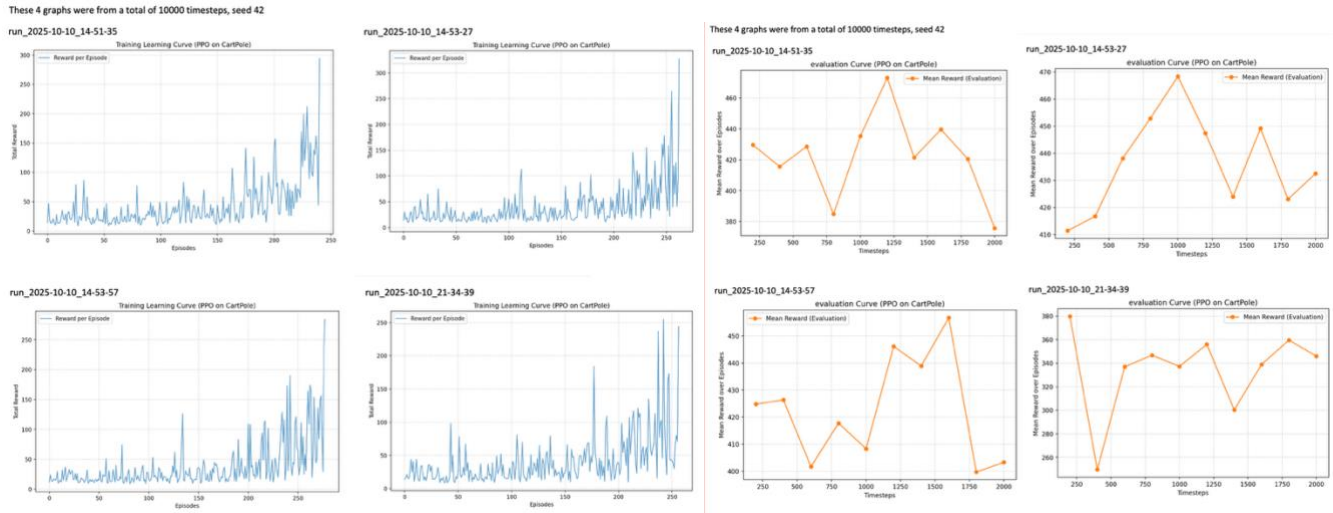

Figure 9-1 PPO Training graphs on the left, evaluation graphs on the right for the CartPole environment

The consistent shape across all four runs, with similar upward trends and convergence points demonstrates PPO's reproducibility. This comes down to the clipped policy gradient, which prevents overly large updates and produces gradual, stable growth rather than unpredictable spikes [2]. The 20,000-timestep run approaches maximum reward (500), confirming that extended training continues to refine an already-converged policy.

On FrozenLake-v1 (100,000 timesteps), the training curve shows PPO beginning with sparse rewards of 0 in early episodes as it explores randomly. There are occasional spikes to 1 as the correct path is discovered by chance. From around episode 2,000-6,000 the frequency of reward spikes increases as the policy updates converge toward the optimal trajectory. By episodes 10,000-16,000 the agent stabilises near a reward of 1 consistently, demonstrating full convergence. The evaluation curve confirms this: mean reward reaches 1.0 early and remains perfectly flat across all 100,000 timesteps, with no variance between runs. Reflects the deterministic nature of the environment and PPO's ability to retain a converged policy without degradation.

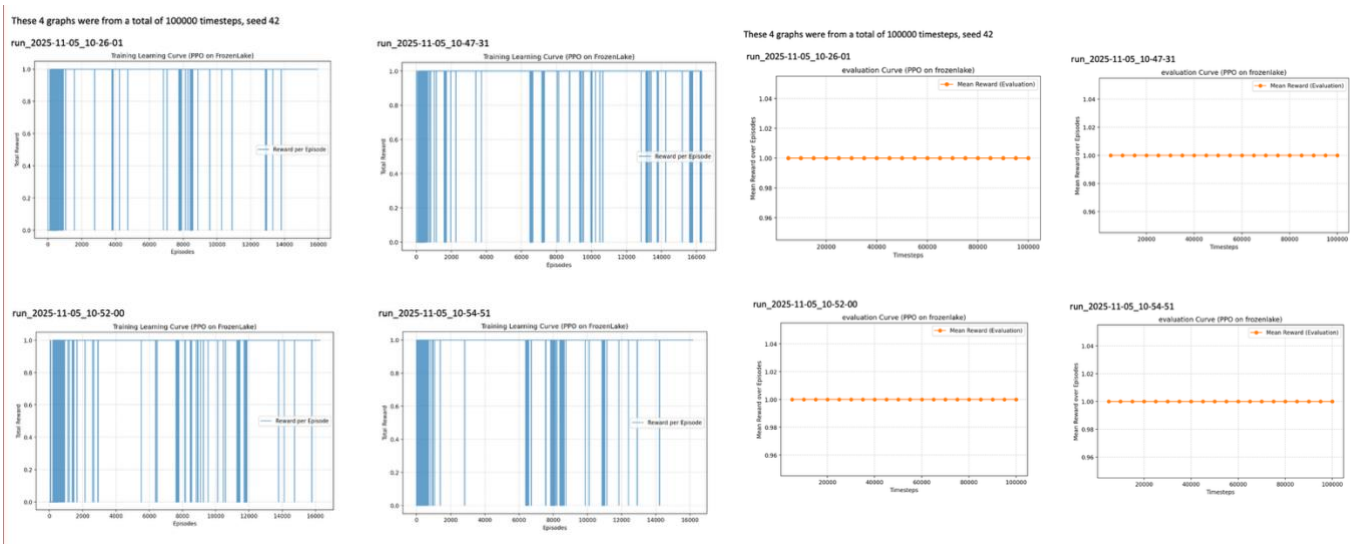

Figure 9-2 PPO Training graphs on the left, evaluation graphs on the right for the FrozenLake environment

9.2 CartPole-v1 & FrozenLake-v1 - DQN

DQN shows a very different training profile from PPO on CartPole-v1. In episodes 0-1,500, rewards are very low and scattered in the 0-80 range. This is because DQN relies on its replay buffer for learning. Early on the buffer contains no useful patterns yet, since the epsilon-greedy strategy is still populating it with random transitions rather than learning from them. The drop around episode 500 reflects instability as the Q-network samples past experiences but cannot yet correlate states to optimal actions.

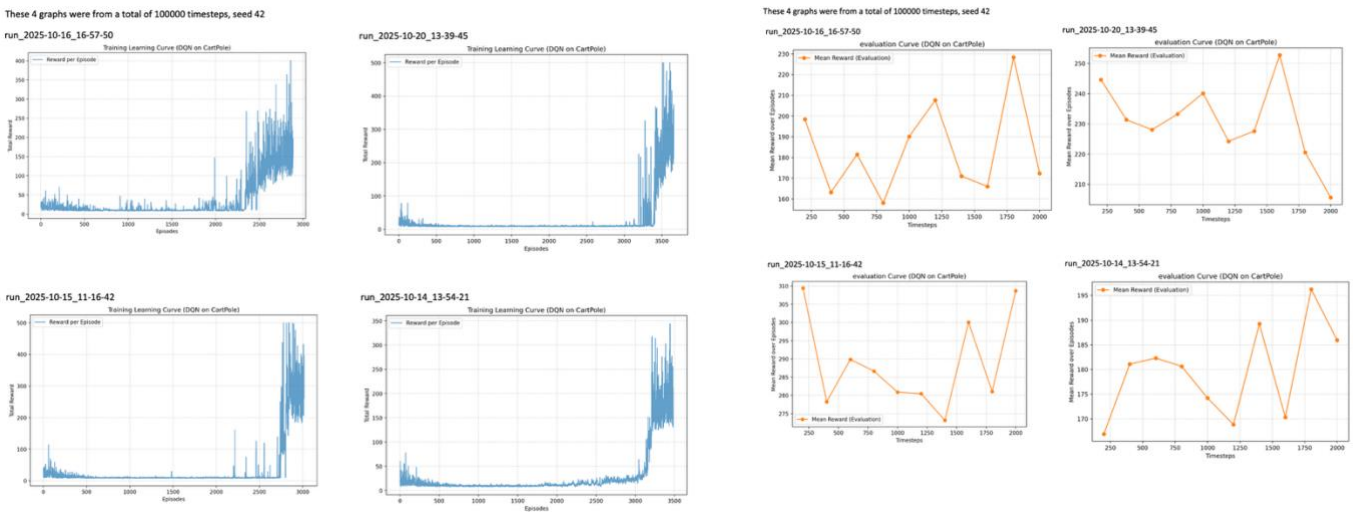

Figure 9-3 DQN Training graphs on the left, evaluation graphs on the right for the CartPole environment

Around episodes 2,000-2,800, the first sustained reward spikes appear as the Q-network begins to generalise across states. The replay buffer is now diverse enough that Q-value updates start reinforcing beneficial actions. Epsilon-greedy decay also reduces the exploration rate here, biasing the agent toward exploitation of learned Q-values. From episode 2,800 onwards, rewards rise sharply to 200-500 per episode. The jagged appearance is characteristic of DQN. Unlike PPO's smooth gradient updates, DQN uses discrete Q-target jumps with random replay sampling, causing noisy reward patterns [3].

On FrozenLake-v1 (100,000 timesteps), the training curve shows high variance early on as DQN fills its replay buffer with random transitions. Causes rewards to fluctuate heavily between 0 and 1 with no stable pattern. From around episodes 1,000-3,000, the Q-network begins generalising correctly and reward spikes become denser and more frequent. By episodes 6,000-10,000+ the agent stabilises with near-consistent rewards of 1, though with heavy epsilon-greedy noise. The evaluation curve shows DQN converging to a mean reward of 1.0 early and holding it perfectly flat across all 100,000 timesteps. This shows complete policy convergence and strong retention in a deterministic environment, despite the slower and noisier path to get there compared to PPO and A2C.

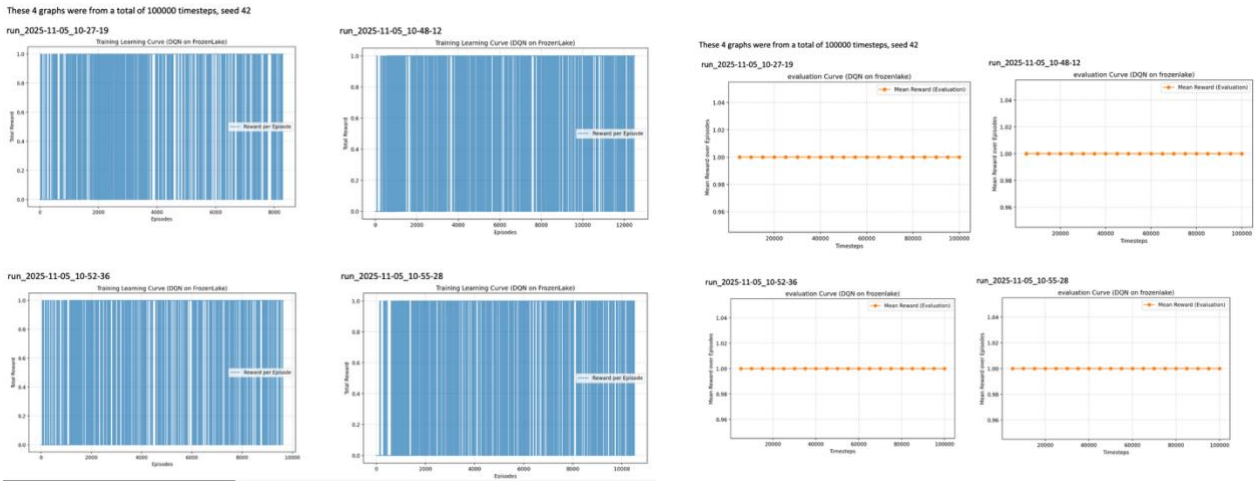

Figure 9-4 DQN Training graphs on the left, evaluation graphs on the right for the FrozenLake environment

9.3 FrozenLake-v1 - A2C

A2C was evaluated on FrozenLake-v1 only (100,000 timesteps), achieving the fastest convergence of the three algorithms. The training curve shows rewards beginning at 0 as the agent explores randomly, with occasional successful completions appearing early. From around episodes 1,000-2,500, reward spikes become more frequent as the actor-critic structure uses advantage estimates to identify and reinforce the correct sequence of actions. By episodes 2,500-5,000 the agent is achieving near-consistent completions, and from episode 5,000 onwards the policy is fully converged, with rewards of 1 across long stretches and minimal variation.

The evaluation curve is the strongest single result across all three algorithms. Mean reward is fixed at 1.0 from the very first evaluation checkpoint and remains perfectly flat across all 100,000 timesteps, with tiny variance between any of the four runs. A2C discovers the optimal deterministic path almost immediately because its parallel actor updates stabilise around the optimum without noise. This demonstrates A2C's capability for fast, stable convergence in deterministic sparse-reward environments, though it was not selected for Phase 2 due to PPO's superior scalability to multi-agent settings.

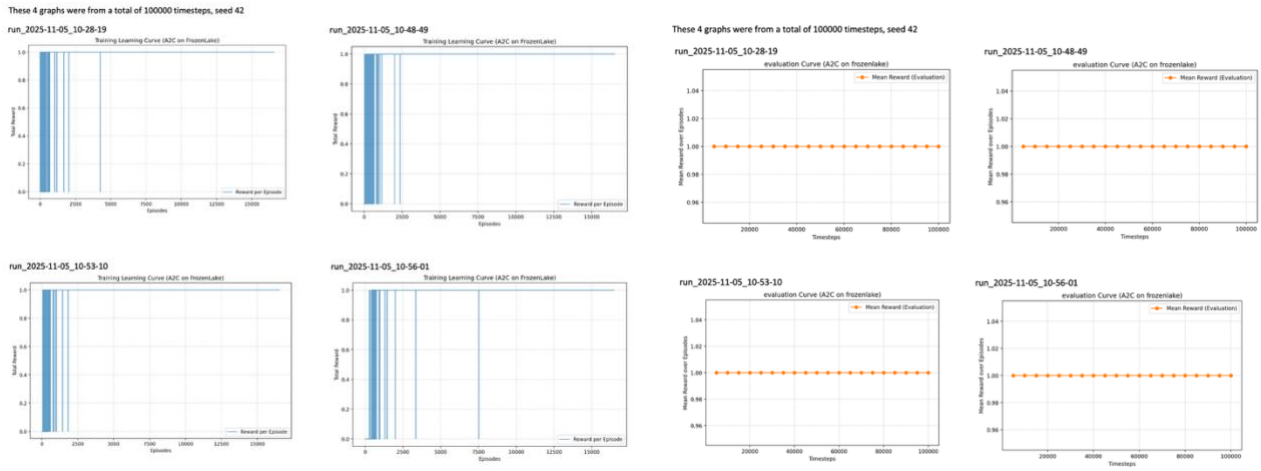

Figure 9-5 A2C Training graphs on the left, evaluation graphs on the right for the FrozenLake environment

9.4 Cross-Algorithm Comparison

The FrozenLake-v1 comparison (is_slippery=False, 100,000 timesteps) gave the following results:

Algorithm	Success Rate	Convergence	Stability	Key Characteristic
PPO	~97%	~3,000 eps	High	Clipped updates, smooth but slower
A2C	~98%	~2,000 eps	High, rare drops	Fastest convergence, parallel workers
DQN	~83%	~5,000 eps	Moderate	Slower start, replay buffer dependent
Baseline	~0%	N/A	N/A	Random actions, no learning

The ranking A2C > PPO > DQN >> Baseline was observed consistently across all runs. Both PPO and A2C converged within the first few thousand episodes and reached near-maximum performance. DQN converged but with higher early variance before stabilising around 0.8-0.9 mean reward in training, reflecting its dependence on replay buffer population. That said, the evaluation curves for all three algorithms show perfect flat-line convergence at mean reward 1.0 once trained, confirming that all algorithms ultimately find the optimal policy in this deterministic environment.

An important finding during FrozenLake testing was the effect of the is_slippery parameter. Initial runs used the default is_slippery=True setting, which introduces random slip probability. DQN in particular produced very low rewards that appeared to be a bug. A meeting with my project supervisor confirmed that is_slippery=False is the correct setting for this project, as the

deterministic version is more analogous to the CybORG environment where actions have deterministic outcomes. All final FrozenLake results use is_slippery=False.

Based on these results, PPO was chosen for Phase 2. It offers the best balance of stability and performance, works well for episodic defence problems, and scales well to multi-agent scenarios. With multiple independent CybORG blue agents each defending a different subnet, it only made sense to choose PPO. A2C, while converging fastest, doesn't support stable multi-agent parallelism in the Stable-Baselines3 implementation used. DQN was ruled out due to its lower sample efficiency and less stable training behaviour.

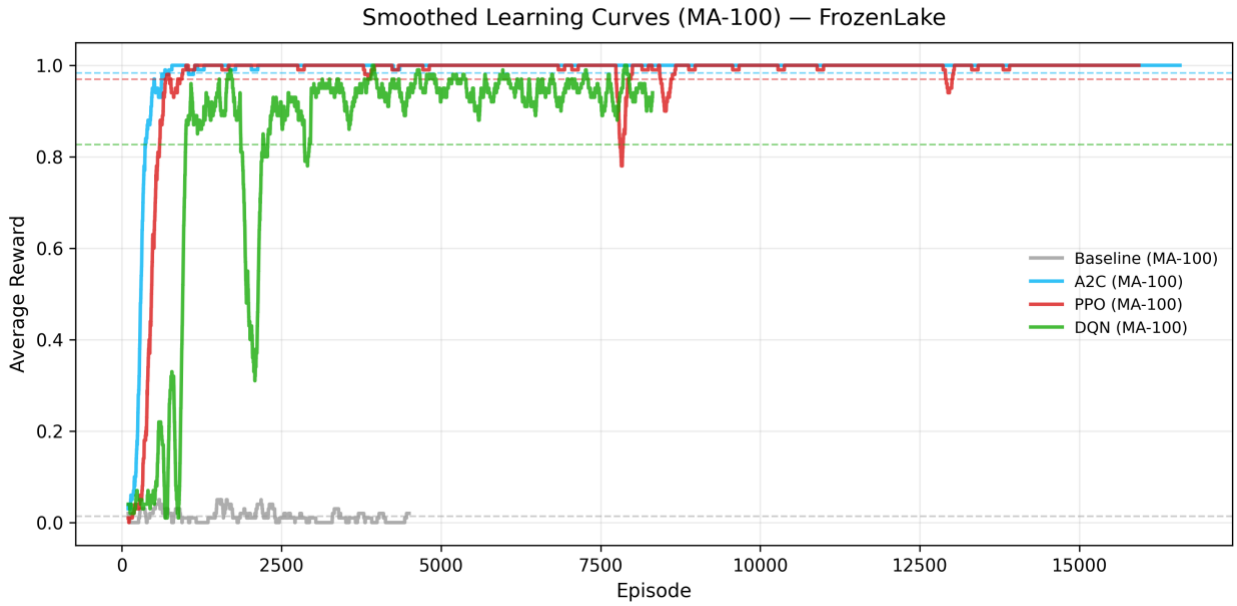

Figure 9-6 A 100-tab moving average smoothed learning curve showing the rewards per episode of the 3 algorithms

10 Phase 2: CybORG Simulation Results

Following Phase 1, the trained GNN-PPO agent (from the Cybermonic CAGE 4 competition submission [19], used for evaluation and bridge adaptation) was evaluated in the CybORG Cage 4 simulation against three red agent types: SleepAgent (baseline), FiniteStateRedAgent (structured attacker), and RedMeanderAgent (random attacker). 50 episodes per agent were run. In CybORG, rewards are always negative: a reward of 0 per step would represent perfect defence at zero cost, which is unachievable since defensive actions themselves carry a cost.

10.1 Cumulative Rewards

Cumulative reward plots reveal the aggregate penalty accumulated over 50 episodes. The SleepAgent baseline (green) accumulated approximately -2,500 total, producing a very gentle slope consistent with only defensive action costs and no real attack pressure. This is the theoretical lower bound on how much penalty blue can accumulate even against a nothing burger opponent.

The FiniteStateRedAgent (red) reached approximately -9,000 cumulative reward. Importantly, the slope is linear and consistent, indicating the blue agent is holding the attacker to a roughly constant level of damage per episode. The situation isn't getting progressively worse, suggesting the agent has learned a partial counter-strategy. The ~6,500 gap between Sleep and FiniteState is the overhead cost of a competent structured attacker.

The RedMeanderAgent (orange) reached approximately -12,500 cumulative reward, producing the steepest slope with the most variable trajectory. Random attackers occasionally chain together highly damaging combinations that the blue agent has no learned response to, causing episodic spikes. The extra ~3,500 cumulative penalty over FiniteState reflects the cost of unpredictability.

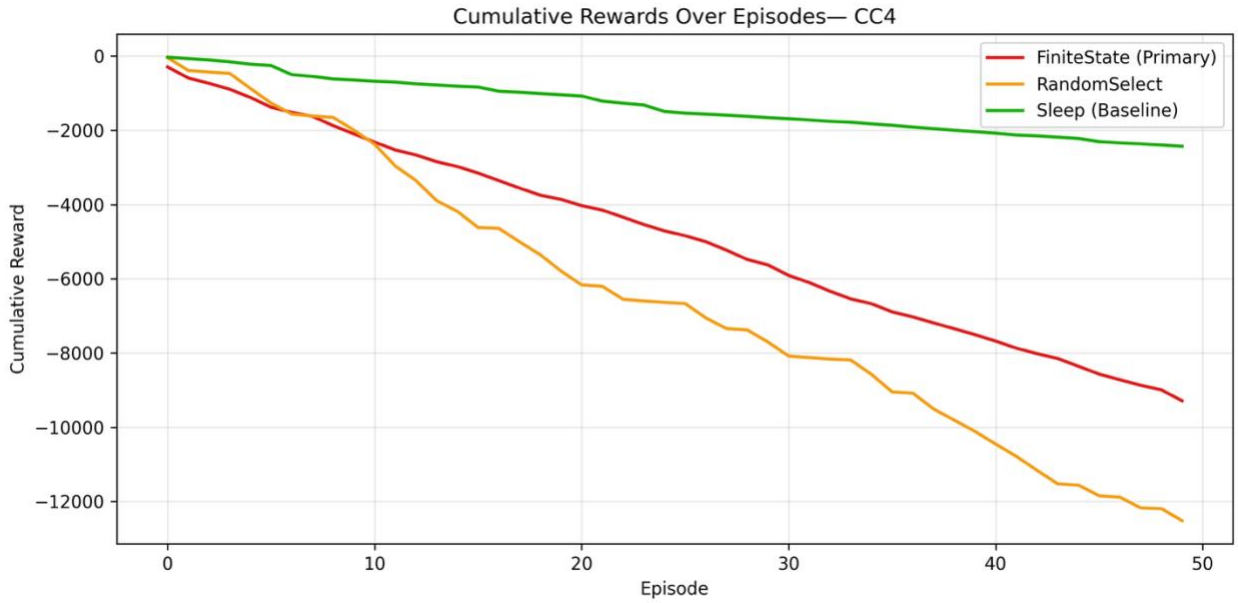

Figure 10-1 GNN-PPO Agent Cumulative Rewards per episode against the 3 Red Agent Types in CybORG Simulation- FiniteState Red Agent, Random Select Red Agent, Sleep Red Agent

10.2 Per-Episode Performance

Mean episode rewards: Sleep ~-40 to -50, FiniteState ~-180 to -200, RandomSelect ~-250. These confirm the cumulative picture. The minimum episode rewards are more revealing however. Sleep stays near -50, FiniteState's worst episode reaches ~-300, and RandomSelect approaches -600 in its worst episode, representing a catastrophic complete collapse. This worst case for RandomSelect occurs when it randomly chains actions that compromise multiple hosts simultaneously, overwhelming the blue agent's single-step response capacity.

The Interquartile Range (IQR) quantifies consistency: Sleep IQR = 14.8 (extremely consistent), FiniteState IQR = 71.2 (structured variance showing genuine engagement with threats), RandomSelect IQR = 337.8 (23x Sleep, essentially lottery-like performance). The Coefficient of Variation confirms this: FiniteState achieves the lowest value (26.7%) despite being the most challenging attacker, confirming the agent has developed a stable counter-strategy specifically against structured attacks. Critically, episodes below -300: Sleep = 0, FiniteState = 0, RandomSelect = 27 out of 50 (54%). The agent never catastrophically fails against a structured attacker.

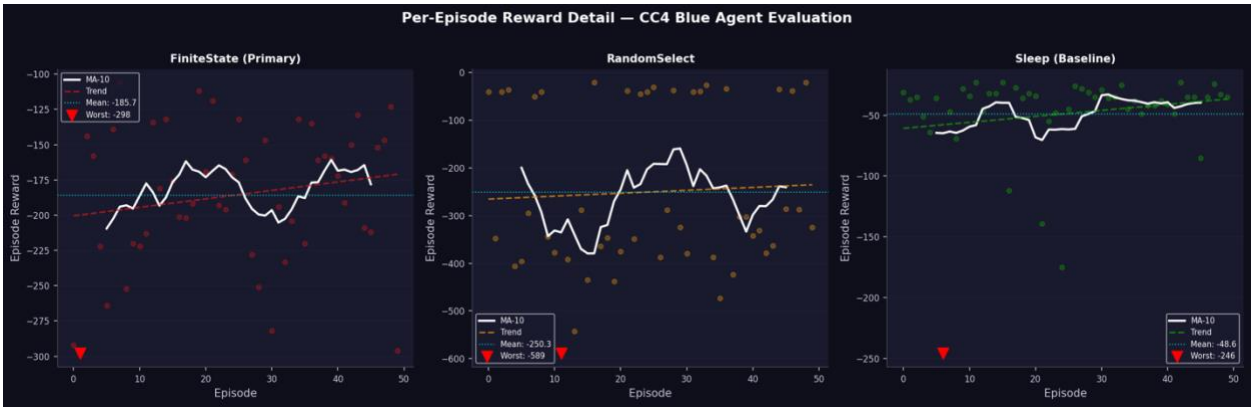

Figure 10-2 of GNN-PPO Agent against the 3 Red Agent Types in CybORG Simulation

10.3 Smoothed Learning Curves

The MA-20 smoothed learning curves show whether the agent is genuinely learning across the 50-episode evaluation window. Against SleepAgent, the curve is flat near -40 to -50 since there is nothing to learn from. Against FiniteStateRedAgent, the agent starts poorly at approximately -300 in early episodes but improves significantly by episodes 10-15, stabilising in the -175 to -190 range. This convergence plateau is encouraging. It shows the agent has found a partial counter-strategy but has hit a ceiling beyond which the remaining ~-180 per episode represents the irreducible cost of FiniteState's structured attacks.

Against RandomSelect, no upward trend is visible. The moving average stays volatile in the -230 to -280 range throughout all 50 episodes. This lines up with the fundamental limitation: there’s no stable pattern to learn from a purely random opponent. The spike around episodes 3-5 followed by a crash is explained by RandomSelect happening to draw low-impact random actions in those episodes, not genuine policy improvement. First-half vs second-half analysis confirms this: FiniteState improves +5 per episode (policy has converged), RandomSelect shows +30 (noise, not learning).

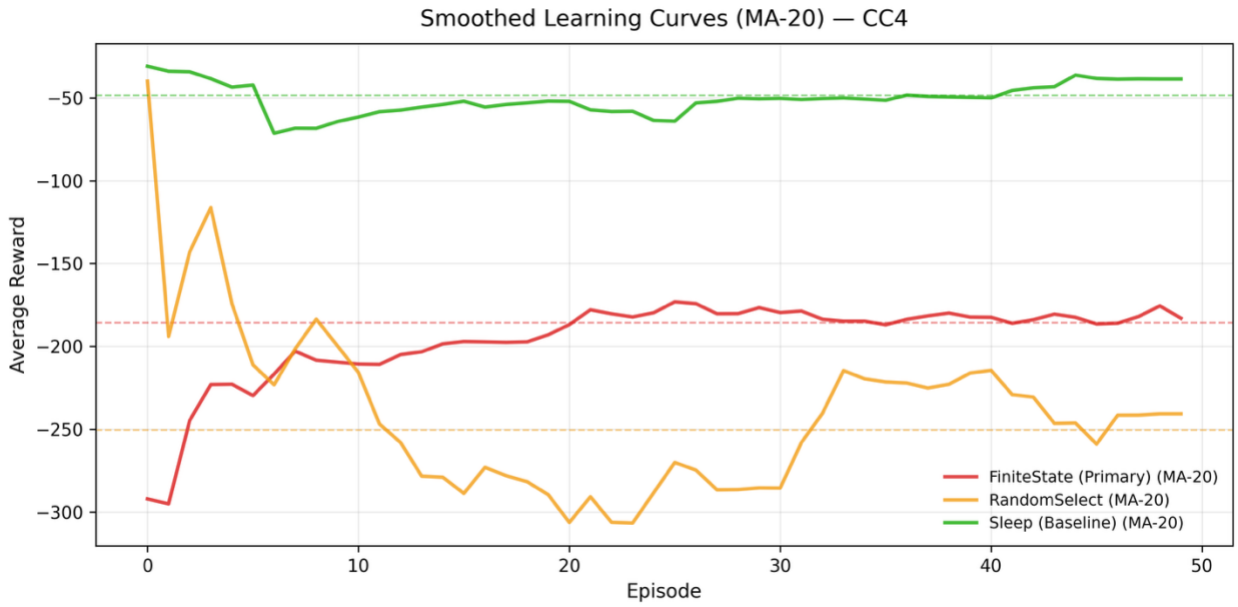

Figure 10-3 A 20-tab moving average smoothed learning curve showing the rewards per episode of the GNN-PPO Agent against the three Red agent types in the CybORG Simulation

10.4 Action Distribution Analysis

The action distribution heatmap reveals what the blue agent is doing rather than just how well it performs. The dominant actions across all scenarios are Monitor (~33-38%) and AllowTrafficZone (~27-30%). Together they account for approximately 60% of all actions. The key finding here is that this distribution is nearly identical regardless of which red agent is attacking, meaning the policy isn't adapting to threat level. Against RandomSelect, the most dangerous opponent, the agent chooses the same proportion of Monitor, Restore, and Analyse actions as against a do-nothing opponent.

This action rigidity is the behavioural explanation for RandomSelect's performance collapse: the agent can't dynamically shift toward more aggressive remediation (Restore, Remove) when under heavy multi-vector attack. This highlights the need for the need for training against diverse and stochastic attackers, not just FiniteState-style structured sequences.

11 Bridge Development and Sim-to-Real Transfer

11.1 Containerlab Network Topology

The Containerlab [9] network topology was deployed on a Mac using OrbStack [25] [26]. Orbstack is a lightweight macOS tool that provides a full Linux VM environment with Docker support, significantly faster than Docker Desktop. This is used as Linux is not natively available on macOS. OrbStack also automatically bridges ports between the Linux VM and the macOS host, meaning anything running on localhost:8080 inside OrbStack is directly accessible in the macOS browser with no manual port forwarding.

The topology file (cage4-topology.yaml) defined using Containerlabs YAML schema [24], implements a representative subset of CybORG CAGE 4's zone structure, using the same nine

subnet names and node roles (restricted-a, restricted-b, operational-a, operational-b, contractor, public-access, admin, office, and internet), each with dedicated routers, servers, and user hosts. There is however a reduced host count to keep the deployment tractable on a single machine. This intentional simplification also functions as a test of generalisation: whether a policy trained on the full simulation transfers to a structurally similar but smaller network. An internet router container handles inter-subnet routing via static route configuration. The episode loop (main.py) ran under sudo inside the OrbStack Linux VM, with a native Linux Python venv providing PyTorch, Docker SDK, NumPy, and PyTorch Geometric.

One thing that came up during CI/CD integration was that the GitHub Actions pipeline, which spins up containers using Docker Compose with the clab prefix, discovered containers almost identically to the OrbStack setup. This was useful as it confirmed the monitor's container discovery logic is portable across Docker environments with minimal changes.

11.2 Baseline Deployment - No Action Masking

The agent was first deployed without action masking, allowing it to select freely from all 81 actions. Included edge actions 64-79 (firewall rules) which are essentially ineffective in Containerlab. This baseline run establishes the raw sim-to-real gap before any mitigation.

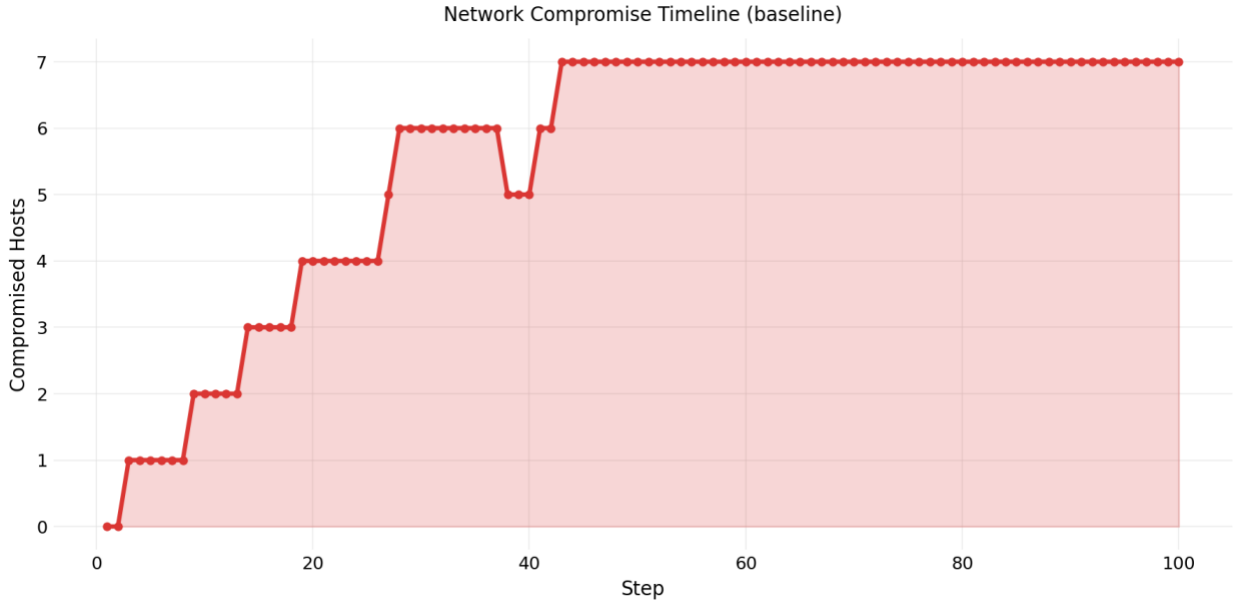

Figure 11-1 Compromised host count for the live network over episodes - unmasked GNN-PPO Agent

The compromise timeline shows a near-monotonically rising curve, with compromise levels across hosts increasing steadily throughout the 100-step. The blue agent is failing to hold the line.

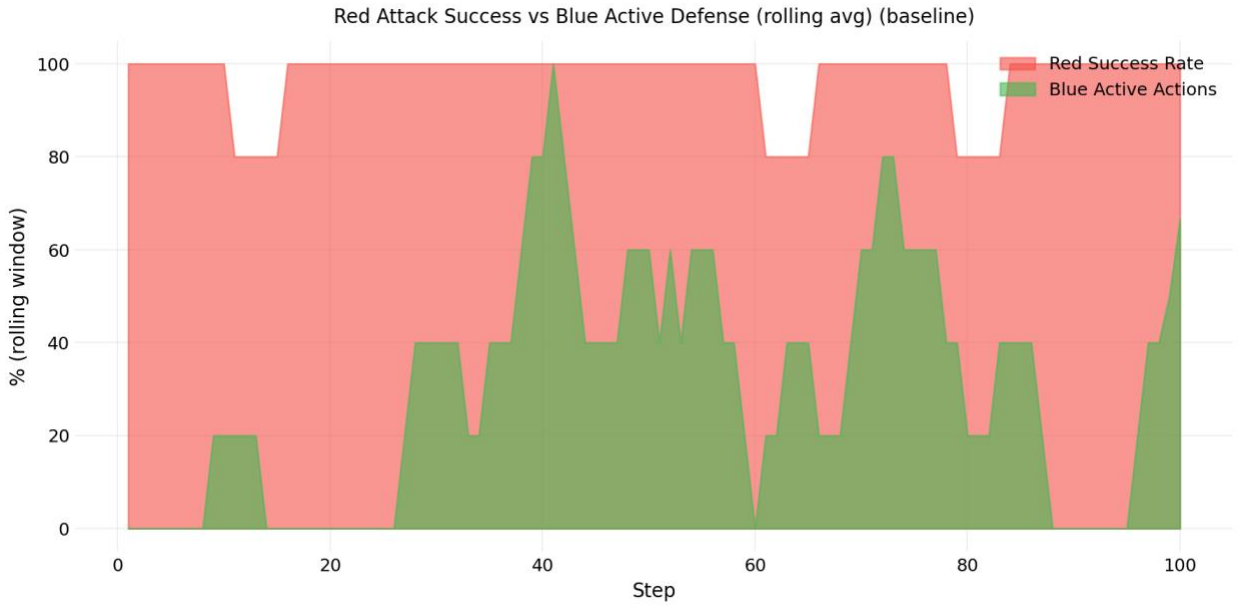

Figure 11-2 Red success v Blue defensive actions success rate on live network over episodes - unmasked GNN-PPO Agent

The red/blue action timeline shows the blue active response area (green) is minimal and flat. Action distribution analysis confirms this: edge actions (AllowTrafficZone and BlockTrafficZone) account for 69% of all selections in the unmasked bridge run, compared to 51.5% in CybORG simulation. Every one of these steps is wasted, as the commands execute but have no observable effect on compromise state because the red agent bypasses the Containerlab topology subnet-level firewall enforcement. Pure Monitor accounts for only 9% of actions, with Analyse at 11% and Remove at 5%.

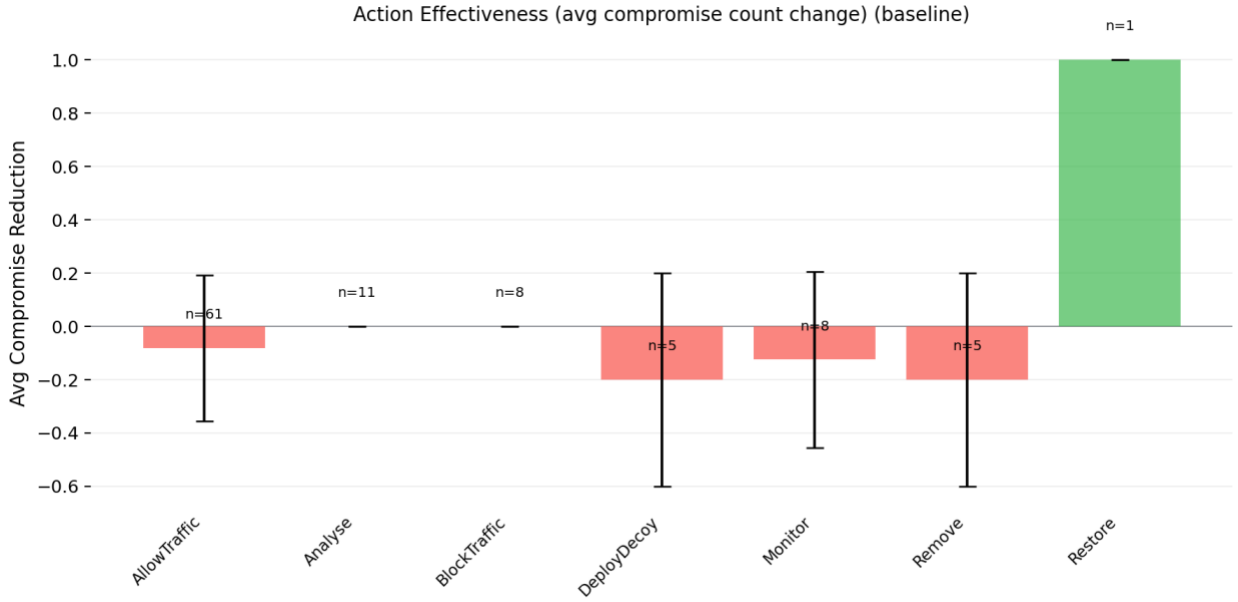

Figure 11-3 Change in compromise count based on GNN-PPO action choices - unmasked

The action effectiveness plot confirms this: Restore shows near-zero or negative impact because the agent selects it infrequently and inconsistently. The dominant actions are edge no-ops which produce no observable change in compromise state.

This baseline behaviour is explained by two factors. First, 16 of 81 action slots (~20%) map to firewall operations that are ineffective in Containerlab. The agent selects these even more heavily in the bridge (69%) than in simulation (51.5%), because the sparser observation compresses its confidence and pushes probability mass toward the actions it was most frequently rewarded for in training. Second, the feature vector is sparser than training: simulation-only metadata (OS version, service pack, file system state) is zero in the bridge. By compressing the agent's probability distribution toward lower-confidence actions, including the edge no-ops, it was trained to select these in certain states.

11.3 Masked Deployment - Inference-Time Action Masking

The agent was then deployed with inference-time action masking applied: edge action probabilities (indices 64-79) are zeroed, and the remaining distribution renormalised before sampling. The trained weights are unchanged and only the deployment-time action selection is constrained. This follows standard deployment practice for RL agents operating in environments with a reduced feasible action set [12][13] [14] [15] [16].

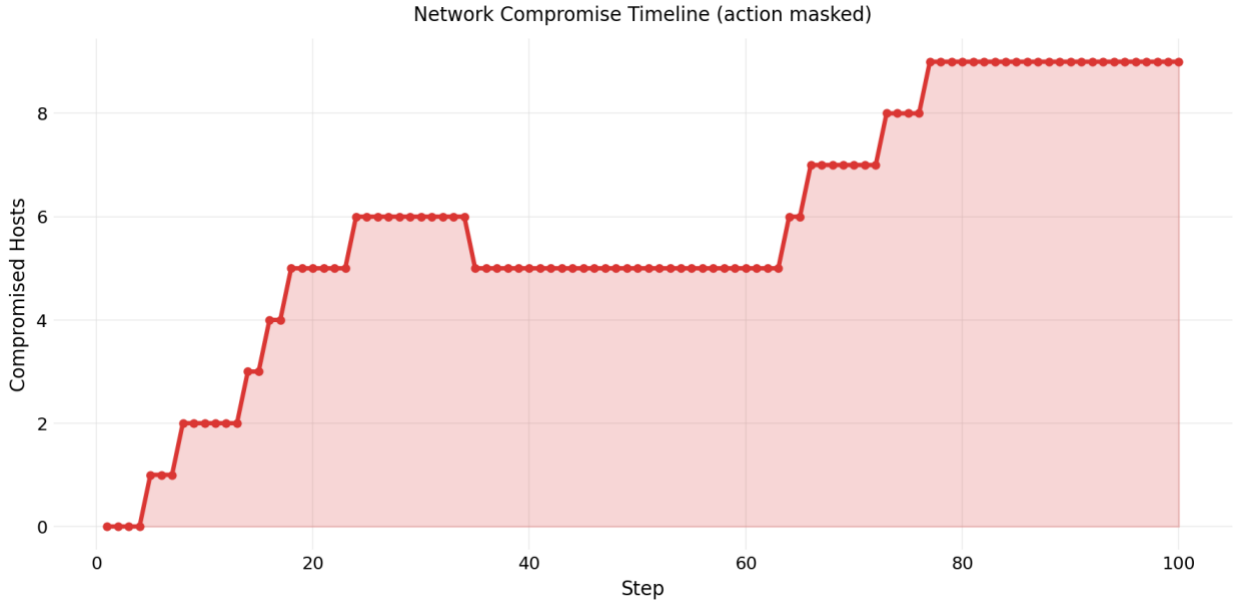

Figure 11-4 Compromised host count for the live network over episodes - masked action GNN-PPO Agent

The compromise timeline shows visible flat sections and dips, reflecting periods where the blue agent is successfully holding or reducing compromise. The curve rises monotonically, but indicates active and effective defence.

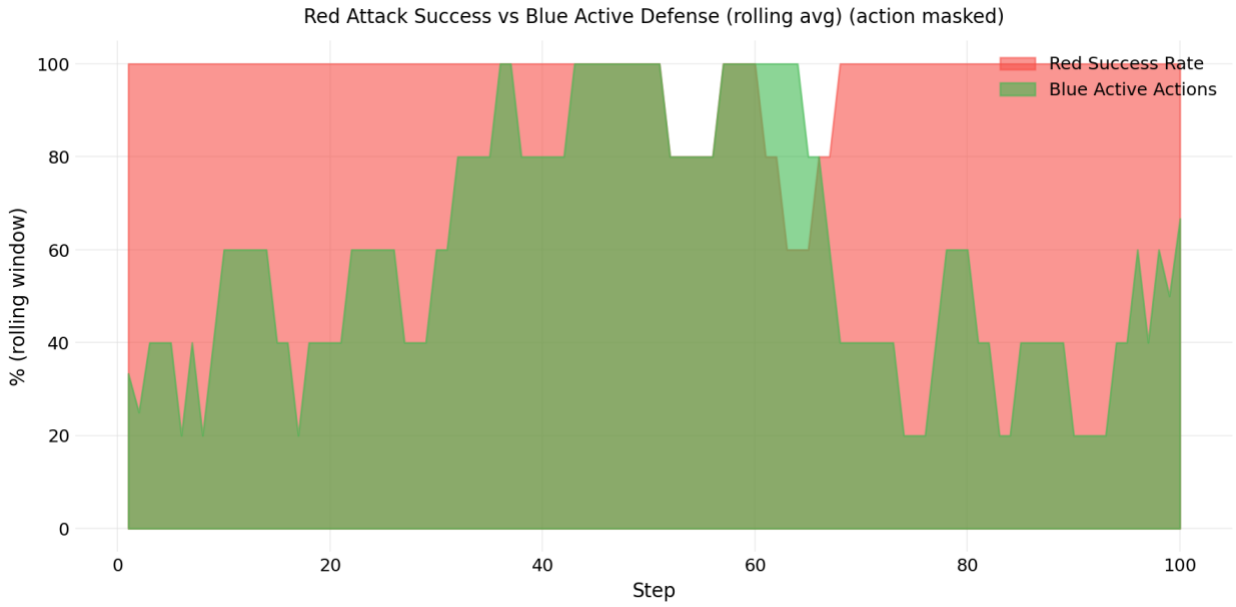

Figure 11-5 Red success v Blue defensive actions success rate on live network over episodes - masked GNN-PPO Agent

The red/blue action timeline shows a substantially larger and fluctuating green area. With edge actions removed, the agent's action distribution shifts significantly: Monitor accounts for 41% of selections, Analyse 39%, Remove 10%, and DeployDecoy 9%. The agent is actively responding across the episode rather than defaulting to passive no-ops. The fluctuation reflects genuine back-and-forth between red attacks and blue remediation.

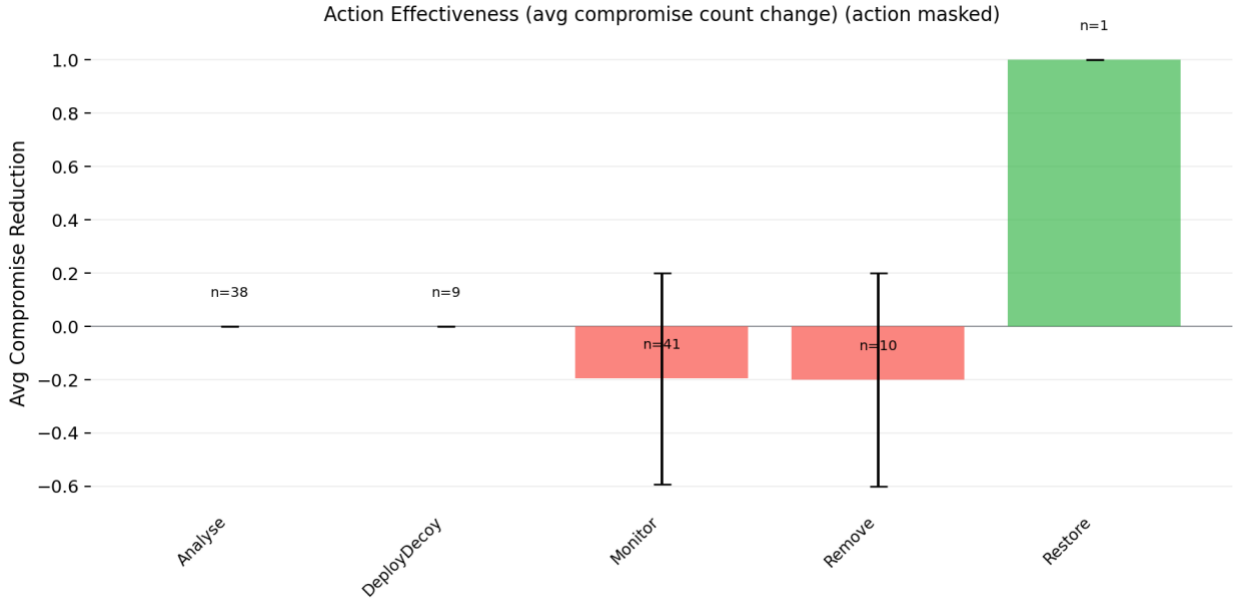

Figure 11-6 Change in compromise count based on GNN-PPO action choices - masked

The action effectiveness plot shows Restore with a positive green bar, confirming remediation is working after the root marker fix. Analyse is the most frequently selected

defensive action (39%) but shows slightly negative effectiveness, which is correct. Analysing a host does not remove a compromise, it only informs the next decision. The increase in Restores and DecoyDeploys shows positive effectiveness when selected. This shows consistency with decoy trapping the red agent, and restoring compromised hosts.

11.4 Action Distribution Comparison -Simulation vs Bridge

The action distribution comparison across all three conditions (CybORG simulation, unmasked bridge, and masked bridge) provides the core policy transfer validation:

Action	**CybORG**	**Bridge (Unmasked)**	**Bridge (Masked)**
Monitor	34.3%	9%	41%
AllowTrafficZone	44.4%	61%	0%
BlockTrafficZone	7.1%	8%	0%
Edge total	**51.5%**	**69%**	**0%**
Analyse	3%	11%	39%
Restore	6.1%	1%	1%
DeployDecoy	2%	5%	9%
Remove	3%	5%	10%
Active total	**14.1%**	**22%**	**59%**

In CybORG, the agent is passive 85.8% of the time, split between Monitor (34.3%) and edge actions (51.5%). Without masking, the bridge is even more edge-heavy: 69% of all selections are AllowTrafficZone or BlockTrafficZone. Confirming that the sparser observation pushes the agent further toward these no-ops rather than correcting away from them. Every one of these steps is wasted.

With masking applied, edge selections drop to zero as expected. The Monitor rate transfers relatively close at 34.3% in CybORG versus 41% in the masked bridge, confirming the agent's fundamental watch-then-act instinct is preserved. The biggest change is Analyse rising from 3% in simulation to 39% in the masked bridge. This is because of observation sparsity. With simulation-only metadata absent from the feature vector, the agent is less certain about host state and probes more before committing to remediation. This is rational behaviour under uncertainty since Analyse is cheap and informative, making it the sensible choice when observations are incomplete.

Remove increases from 3% to 10% and DeployDecoy from 2% to 9%, both absorbing the probability mass previously assigned to edge actions. Restore remains low in both bridge conditions (1%) compared to simulation (6.1%), reflecting lower confidence about when a host is compromised enough to warrant a full restart. The overall active action rate rises from 14.1% in

CybORG to 59% in the masked bridge. Agent has become more of an interventionist when constrained to executable actions only.

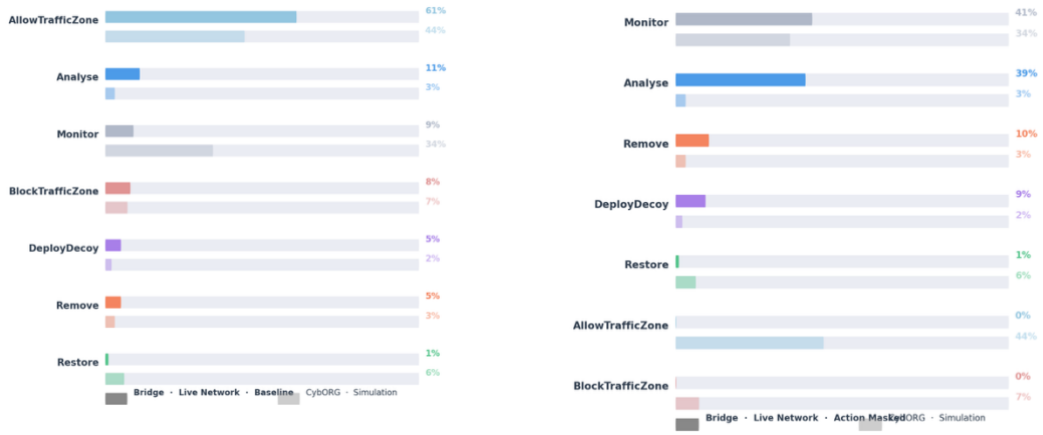

Figure 11-7 Unmasked v simulation action distribution chart on the left, Masked v simulation action distribution chart on the right

11.5 Sim-to-Real Gap Analysis

The baseline and masked results together characterise the sim-to-real gap for this deployment.

Action feasibility gap: 16/81 action slots (~20%) are structurally valid in CybORG but unsupported in Containerlab. In the unmasked bridge, these account for 69% of all selections, which is higher than the 51.5% in simulation, because the sparser observation amplifies the agent's preference for these actions rather than correcting it. This gap is fully resolved by inference-time masking, which prevents invalid action selection without modifying the trained weights.

Observation sparsity gap: CybORG tracks rich simulation metadata per host (OS version, service pack, file system state, kernel version) encoded across indices 6-53 and 103-177 of the 192-dimensional feature vector. These are zero in the bridge because Docker containers don't expose equivalent metadata. The clearest sign of this gap is the Analyse rate rising from 3% in simulation to 39% in the masked bridge. The agent probes more because it cannot read host state as confidently as it could in training. This gap is partially mitigated by populating all observable features from live container data but can't be fully closed without retraining on bridge observations [11].

Topology scale gap: The bridge topology implements a representative subset of CybORG CAGE 4's zone structure with a reduced host count. The consistent Monitor rate (34% vs 41%) across environments despite this difference suggests the GNN's graph-based representation generalises across network scales.

What transferred correctly: Despite all three gaps, the agent's fundamental strategy is preserved. Monitor rates are consistent (34% simulation vs 41% bridge). The agent does not collapse to all-Monitor or behave randomly when deployed outside simulation. The differences in Analyse and Remove are explained by known environment constraints, not policy breakdown. The masked bridge results show that the trained GNN-PPO policy generalises to real containerised infrastructure, with the action feasibility gap being the primary obstacle to full deployment parity and inference-time masking its practical resolution.

12 Problems & Solutions

This section covers the technical problems encountered across both semesters of the project and how each was resolved. Problems are grouped by phase.

12.1 Phase 1 - Algorithm Benchmarking

12.1.1 Laptop Overheating During Training

During Sprint 2, attempting to run the PPO training curve script locally caused the laptop to overheat and required cancellation after ten minutes. The script was designed to train the model and record video demos simultaneously, adding significant CPU overhead.

Resolution: The training and video recording logic was separated into two independent scripts. The model was trained first and saved as a .zip checkpoint, then videos were recorded by loading the saved weights in a separate pass. This also clarified the conceptual distinction between a training curve (rewards recorded during training via Monitor) and an evaluation curve (rewards from a frozen policy loaded deterministically). The issue was also addressed by using the college computer lab (room 507/508) when training runs required extended time.

12.1.2 Jupyter Kernel Not Recognised on College Computer

When attempting to run Jupyter notebooks on the college Windows computers, the notebooks failed to recognise the Python kernel, producing a 'kernel not found' error.

Resolution: Downgrading the Jupyter extension in VS Code to an older version resolved the issue. A longer-term fix was to standardise the working environment by using the same virtual environment and interpreter path across both the Mac and college Windows machines.

12.1.3 FrozenLake Graph Bug - Reward Threshold Set to 475

When comparing algorithm performance on FrozenLake, the comparison script contained a hardcoded success threshold of 475, copied over from CartPole where the maximum reward is 500. FrozenLake rewards are binary (0 or 1), so a threshold of 475 resulted in a 0% success rate for all algorithms even when they were solving the environment correctly.

Resolution: The threshold was corrected to 0 (any non-zero reward = success). This fixed the comparison graph which had previously shown all algorithms at 0% success rate. CartPole and FrozenLake require separate metric logic because their reward structures are completely different.

12.1.4 PPO Timestep Count Off by 240

When training PPO with total_timesteps=10000, the final logged timestep count was 10240 rather than 10000. This initially looked like a potential bug in the training code.

Resolution: This is expected behaviour. PPO collects data in chunks of 2048 (the n_steps parameter) and rounds up to the nearest multiple: 2048 × 5 = 10240. This confirmed understanding of how Stable-Baselines3 PPO works internally and was documented in the project OneNote.

12.1.5 CI/CD Branch Push Logic Misunderstood

Early in the project, commits were being pushed from feature branches directly to main. This violated the intended Git Flow where feature branches merge to develop, and only release-ready code goes to main.

Resolution: The GitHub Actions CI pipeline was updated to trigger on pushes to develop and feature/** branches, and on pull requests to main. The Jira SCRUM branch naming convention (SCRUM-**-feature/**) was also added, aligning the project with standard Git Flow practice.

12.1.6 Is_slippery = True - Producing Poor FrozenLake Results

Initial FrozenLake runs used the default is_slippery=True setting, which introduces random slip probability. DQN was producing very low rewards that looked like a bug, and A2C was less stable than expected.

Resolution: Supervisor confirmed that is_slippery=False is the appropriate setting for this project. Switching to is_slippery=False immediately improved all three algorithms and produced the clean convergence results documented in Section 9.4. All final Phase 1 results use is_slippery=False.

12.2 Phase 2 - CybORG Exploration

12.2.1 CybORG Cage 4 Import Errors - gym vs gymnasium

After installing the CybORG Cage 4 codebase, the entire test suite produced import errors because the code used import gym but the installed package was the newer gymnasium (Gym was deprecated in 2023). This caused AttributeError and ModuleNotFoundError across every file that imported the environment.

Resolution: A global find-and-replace was performed across the entire codebase, changing from gym to from gymnasium and import gym to import gymnasium as gym. After verifying the replacement worked, a develop branch was created and all changes committed. This fixed all import errors without requiring any logic changes.

12.2.2 Training From Scratch - Consistently Degrading Reward

During Week 2 of Semester 2, an attempt was made to train a new PPO agent from scratch on a custom simplified topology (3-zone Mininet network) as an alternative to using the pre-trained CybORG agent. The training ran for 155 iterations (88.2 minutes) before early stopping triggered. The best reward reached was -3,525 and the trend was consistently degrading from iteration 30 onwards, never recovering to the early training high.

Resolution: Supervisor advised that training a capable agent from scratch within the remaining project timeline wasn't practical given computational constraints (Mac CPU only, no GPU). Two alternatives were identified: (1) use inference with the pre-trained Cybermonic GNN-PPO submission agent, or (2) specify a fixed seed so the network topology is consistent across episodes, making training more tractable. The project pivoted to inference, using the Cybermonic agent's weights for all subsequent evaluation and bridge work. This decision is covered further in Section 13 (Design Choices).

12.2.3 Initial Plans for Custom Network - Abandoning Mininet Plans

The original Semester 2 plan was to training an agent using Ray RLlib on a custom Mininet network. Mininet requires Linux, which wasn't available natively on the Mac, so a Multipass Ubuntu VM was set up. After getting Mininet running inside the VM, packet routing across

subnets failed consistently because no router configuration was in place, meaning hosts on different subnets could not ping each other.
Resolution: After discussion with the supervisor, the Mininet approach was abandoned in favour of Containerlab, which uses real Docker containers with proper network bridge support and a well-documented topology definition format, making it a better and more maintainable option for this project.

12.3 Bridge Implementation

12.3.1 Feature Dimension Mismatch

The first version of graph_builder.py set FEATURE_DIM incorrectly, producing node feature tensors of the wrong shape. When the GNN-PPO agent's checkpoint was loaded and a forward pass attempted, PyTorch threw a shape mismatch RuntimeError indicating the agent's first linear layer expected a different input dimension.
Resolution: The trained agent's weights were inspected using a custom inspect_weights.py script which loaded the checkpoint and printed each layer's expected input dimensions. This confirmed the correct FEATURE_DIM should be **192**, matching the full feature vector layout documented in the CAGE 4 training code. FEATURE_DIM was updated to 192 and the feature encoding functions were updated accordingly. This fix was critical since without it the entire bridge pipeline was non-functional.

12.3.2 Agent Blind to Compromise - Feature 187 Never Set

After fixing the feature dimension, the agent was successfully running forward passes and returning action integers. However, it consistently chose Monitor actions even when containers were compromised. Inspecting the feature vectors showed feature index 187 (was_compromised) as 0.0 for every node on every step, even after red had planted compromise markers.
Resolution: The root cause turned out to be a pipeline disconnect. The intrusion detector's compromise_map was being computed correctly but was never being passed into build_graph(). The fix was to pass compromise_map through the adapter.get_action() call signature down to build_graph(). Once wired correctly, feature 187 appeared as 1.0 for compromised nodes and the agent began choosing Restore and Analyse actions on those hosts.

12.3.3 Dead Graph Variable - Compromise Map Not Reaching the Agent

After the first fix for P3.2, a more subtle variant of the same problem was discovered. The main loop called builder.build_graph() once and stored the result, then called adapter.get_action(). But get_action() internally called builder.build_graph() again without the compromise_map, discarding the first graph entirely.
Resolution: The compromise_map was added as an explicit parameter to adapter.get_action(), which forwarded it to its internal build_graph() call. The dead external build_graph() call in main.py was removed. Feature 187 now correctly reached the GNN for every step.

12.3.4 Container Name Resolution - CLAB_PREFIX Mismatch

The first Containerlab topology YAML was named fyp-defense-network, producing container names with the prefix clab-fyp-defense-network-. However, some scripts used a hardcoded

CLAB_PREFIX of clab-cage4-defense-network- inherited from a later topology file rename, causing Docker containers.get() to raise docker.errors.NotFound for every container.
Resolution: The topology was renamed to cage4-defense-network in the YAML, so all containers received the prefix clab-cage4-defense-network-, matching the constant in the bridge code. A single source of truth for the prefix was established in each module.

12.3.5 Block Action - Index Out of Range when Container Already Disconnected

After implementing the BlockTrafficZone action, a crash occurred when the blue agent tried to block a container that was already blocked from a previous step. The _block() method tried to access the container's first Docker network, but an already-disconnected container had an empty Networks dict, causing an IndexError.
Resolution: A check was added to verify whether the container had any networks remaining before attempting disconnection. If Networks was empty, the block was treated as a no-op and logged accordingly.

12.3.6 Stale Compromised Flags Between Test Runs

After running main.py, the /tmp/.compromised and /root/.compromised sentinel files persisted in containers. The next test run would start with false-positive compromises, corrupting results.
Resolution: A cleanup_flags() method was added to IntrusionDetector, called at the start of main.py and evaluate.py before any scanning. In the test suite, a @pytest.fixture(autouse=True) was added to test_intrusion_detector.py that calls cleanup_flags() both before and after every test automatically.

12.3.7 Stale Decoy Containers From Previous Runs

If main.py exited mid-run, decoy containers were left running and would appear in the next monitor scan as unexpected containers, confusing the graph builder's node classification.
Resolution: A cleanup_stale_decoys() function was added to ActionExecutor that ran sudo clab destroy on any decoy topology directories found. This was called at the start of main.py and evaluate.py before the episode loop, and again at graceful shutdown.

12.3.8 Iptables Not Available in Alpine Containers for Edge Actions

When implementing edge actions, the initial plan was to use iptables rules on gateway router containers. Testing showed that iptables was not installed in alpine:latest and could not be installed at runtime because the containers had no working DNS resolution and apk add failed with a DNS transient error.
Resolution: The ip route replace blackhole command was used instead. This is available on all Alpine containers as part of the iproute2 package (pre-installed), requires no internet access, and achieves the same DROP effect for forwarded traffic. A corresponding ip route replace via command was used for AllowTrafficZone to restore the original route.

12.3.9 Edge Action Index Ordering Bug - Docker API vs Alphabetical

The trained GNN-PPO agent builds its edge action space by sorting subnet routers alphabetically (confirmed from reading BlueFixedActionWrapper._populate_action_space() in the CybORG

source). However, in agent_adapter.py, the subnet routers were listed in Docker API return order, which is arbitrary and does not match alphabetical order, meaning edge actions were targeting the wrong subnets.
Resolution: A one-line sort was added to agent_adapter.py: subnet_routers = sorted([r for r in routers if r['clean_name'] != 'internet-router'], key=lambda r: r['clean_name']). This brought the bridge edge ordering into alignment with the training ordering.

12.3.10 Stale .pyc Bytecode - Code Changes Not in Effect

After fixing a method in action_executor.py, the fix was not taking effect despite the file being saved. Repeated testing showed no change.
Resolution: The issue was stale .pyc bytecode cache in the OrbStack macOS mount. Python was running the old compiled bytecode from **pycache** rather than the updated source file. Running rm -rf __pycache__ in the bridge directory before retesting resolved the issue immediately. This was specific to the environment caused by the macOS APFS to Linux ext4 filesystem bridge in OrbStack.

12.3.11 /tmp Persists Across Docker Restarts in Alpine

During comparison evaluation, the Restore action was not lowering the compromise level on the next scan. Investigation revealed that /tmp is not mounted as tmpfs in Alpine Linux containers by default and instead persists across docker restart as part of the container's writable overlay layer.
Resolution: An explicit rm -f /tmp/.compromised was added after the container.restart() call in _restore(). Note: the initial implementation also left /root/.compromised intact on the assumption that root compromise should persist, though this assumption later turned out to be wrong and is documented as a separate bug in P3.17.

12.3.12 DiscoverDeception Bug - Decoy Dict Not Shared

When the blue agent deployed a decoy and the red agent subsequently ran DiscoverDeception on the same host, the red agent always returned False (decoy not found), despite the decoy being visible in executor._decoys.
Resolution: RedAgent was being initialised with decoys={} (a fresh empty dict) rather than executor._decoys. Python dicts are mutable and passed by reference, so passing executor._decoys directly means any decoy the executor adds is immediately visible to the red agent. The fix was a one-line change in main.py: red_agent = RedAgent(containers, decoys=executor._decoys).

12.3.13 Get_last_action() Always Returning Sleep in evaluation.py

When building the CybORG evaluation script, the implementation used cyborg.get_last_action('blue_agent_0') to record what the agent did each step. The output showed 100% Monitor/Sleep across all 100 steps, which was clearly wrong.
Resolution: The GraphWrapper intercepts the agent's action integer and translates it into a CybORG action object, labelling all steps it handles internally as Sleep. The fix was to capture the raw action integer directly from agent.get_action() before it enters wrapped.step(), then decode it manually using the CAGE4 action space formula.

12.3.14 Infinite Restore Loop on Root-Compromised Hosts

During evaluation, the blue agent kept selecting Restore on the same host repeatedly across many consecutive steps with no improvement in compromise state. The intrusion detector continued reporting the host as compromised even after multiple Restore actions.

Root cause: The original _restore() implementation only removed /tmp/.compromised after a container restart, leaving /root/.compromised intact on the assumption that "root access is harder to remediate." This was semantically wrong , in CybORG, Restore is a full re-image (RestoreFromBackup) that removes all attacker sessions. Leaving /root/.compromised intact meant the intrusion detector always saw the host as root-compromised, causing the agent to select Restore again on the next step indefinitely.

Resolution: A single line change in _restore() explicitly removes both sentinel files after container restart: rm -f /tmp/.compromised /root/.compromised. Once this was fixed, the action effectiveness graph showed Restore with a positive green bar (average +0.25 compromise reduction across 4 Restore events), confirming remediation was working correctly.

12.3.15 Sim-to-Real Action Space Gap - Edge Actions as No-Ops

When the full bridge evaluation was run, the blue agent was observed to be mostly passive. The compromise timeline rose steadily with no flat sections, and the red/blue timeline showed almost no active blue response.

Root cause: 16 out of 81 actions (indices 64-79, edge/firewall actions) are no-ops in the bridge because Alpine Linux containers do not have iptables installed and DNS resolution fails inside the data-plane network. Additionally, the feature vector the agent receives is significantly sparser than during training since simulation-only metadata accounts for around 100 of the 192 dimensions and is left as zero in the bridge. Both factors compound: the agent sees an unfamiliar observation and can’t execute ~20% of the actions it might select.

Resolution: Inference-time action masking was applied. The trained weights were kept unchanged. Instead, the Categorical probability distribution produced by the actor network was intercepted before sampling, probabilities at indices 64-79 were zeroed, and the remaining 65 valid action probabilities were renormalised to sum to 1. This is a well-established deployment technique validated in the literature [12], [13]. Argmax was tried first but consistently returned Monitor (80) as the highest single-probability action under a sparse observation, so stochastic sampling was used instead to preserve the policy's behavioural diversity. Once masking was applied, the action effectiveness graph showed Restore with a positive impact on compromise reduction and Analyse as the most frequent active action.

12.3.16 Feature Vector Sparsity in Deployment

The GNN agent's 192-dimensional feature vector was designed for CybORG's simulation environment, which tracks OS versions, file system metadata, service pack levels, and kernel versions. In the bridge, all these dimensions are left as zero because real Alpine Linux containers don’t expose equivalent metadata, compressing the agent's probability distribution toward lower-confidence actions.

Resolution: All observable features were populated from live Docker container data: process detection from ps aux output (SSHD, nginx/apache2, /bin/sh at indices 64, 70, 60, 78), compromise level from sentinel files (feature 187), scanned state from the red FSM host_states

dict (feature 188), decoy flag from executor._decoys (feature 100), and subnet membership from container naming conventions (features 178-186). Simulation-only metadata that cannot be observed from real containers is documented as a known sim-to-real gap, consistent with findings in the sim-to-real transfer literature [11].

13 Design Choices

This section covers the key architectural and technical decisions made during the project, along with the alternatives that were looked at, the reasoning behind each final choice, and what came out of each one.

13.1 RL Environment Selection - CybORG Cage 4 vs Alternatives

The original proposal involved simulating a network using VirtualBox VMs running real network services, with a supervised learning model detecting anomalous traffic as the first stage. This was revised after an initial meeting with the supervisor to focus on Reinforcement Learning in a standardised environment.

Several environments were considered. CybORG Cage 2 (April 2022) [1] was the first candidate looked at, as it has a simpler action space and there is published work comparing agents on it. CybORG Cage 3 (September 2022) was also reviewed. CybORG++ (October 2024) [10] was investigated but ruled out because it is a community fork rather than the official TTCP release, with less documentation. CybORG Cage 4 (February 2024) was ultimately selected because it is the most recent official TTCP challenge, has the richest action space, has an active GitHub repository, and has a published competition submission (Cybermonic [19]) with open-source code and trained weights available.

The Cybermonic GNN-PPO submission was found via a GitHub search for 'cage-4-submission' during Sprint 3 of Semester 2. Using a competition-quality pre-trained agent rather than training from scratch was a significant scope decision, covered further in Section 13.3.

13.2 Network Simulation Tool - Mininet vs Containerlab

Phase 2 originally planned to use Mininet as the realistic network layer, running inside a Multipass Ubuntu VM on the Mac. Mininet was researched because it is a widely used academic tool for software-defined networking research and appeared in supervisor discussions.

After spending time setting up Mininet inside Multipass, it became clear that getting routing to work across subnets was not straightforward. Initial topologies were dropping 66% of packets because no router had been configured between subnets. Containerlab was then investigated as an alternative. Containerlab [9] uses real Docker containers rather than kernel-level network namespaces, has a straightforward YAML topology definition format, integrates well with the Docker Python SDK used in the bridge layer, and any Python tool that works with Docker containers works transparently with Containerlab, whereas Mininet's Python API is more specialised.

The final decision to use Containerlab was confirmed after a supervisor meeting where supervisor endorsed the switch. The Mininet codebase and Multipass VM were abandoned and a new Containerlab topology was designed to mirror CybORG Cage 4's zone structure as closely as possible.

13.3 Bridge Topology - Full CAGE4 Replication vs Simplified Subset

A decision was required about how closely the Containerlab topology should replicate the full CybORG CAGE4 network. The full CAGE4 simulation uses an EnterpriseScenarioGenerator that produces a large dynamic topology with significantly more hosts per subnet than the bridge deployment.

Two approaches were considered. A full replication would match the training environment exactly in terms of host count and subnet sizes, giving the closest possible match to the conditions

the agent was trained in. A simplified subset would use the same nine subnet names and node roles but with a reduced host count, keeping the deployment tractable on a single machine without GPU access or a multi-node Docker cluster.

The simplified subset was chosen for two reasons. First, computational constraints: running the full CAGE4 host count as live Docker containers on a single Mac laptop caused resource contention that made sustained 100-step episodes unreliable. Second, and more academically interesting: deploying on a smaller topology than the agent trained on is a test of generalisation. If the GNN-PPO policy had simply memorised the training topology's structure, it would fail on a different-sized network. The consistent action distributions observed between simulation and bridge (Monitor 34% vs 41%) suggest the agent's graph-based representation generalises across network scales, which is a stronger result than simply matching the topology exactly would have produced.

13.4 Agent Strategy - Train from Scratch vs Inference with Pre-Trained Weights

A critical scope decision was made during Week 2 of Semester 2. The original plan was to train a new GNN-PPO agent on the custom topology. A training script was put together using the CybORG RLlib integration and run for 88.2 minutes (155 iterations). The best reward reached was -3,525, but the trend was degrading consistently from iteration 30 onwards and early stopping triggered at 155 iterations.

Two options were then evaluated: (1) continue training with a fixed random seed so the network topology is consistent across episodes, potentially helping convergence; (2) pivot to inference using the Cybermonic competition submission's pre-trained weights, which had been trained for hundreds of thousands of timesteps with sophisticated curriculum training. Supervisor suggested option 2 as the more tractable path given the computational resources available (Mac M-series CPU, no GPU access).

This decision changed the focus of the project from "train an agent and evaluate it" to "deploy a simulation-trained agent in a real environment and measure policy transfer." This is arguably a more interesting and novel contribution, focusing on sim-to-real transfer [11] for RL-based network defence, and the evaluation results backed this up.

13.5 Frontend Architecture - FastAPI vs stdlib http.server

Nine frontend architecture options were evaluated for the live dashboard: static CSV reader, Flask with state.json, Python's built-in http.server, WebSocket push, Jupyter widget, FastAPI with Jinja2 templates, FastAPI with a background thread, Streamlit, and a React SPA.

FastAPI with Jinja2 was the runner-up and would have provided a cleaner API and proper template rendering. However, it required two additional pip dependencies (fastapi and uvicorn) and involved more moving parts in the deployment. Streamlit was also attractive for its built-in components but its execution model, where Streamlit controls the main thread, conflicts with a running episode loop in main.py.

The final choice was Python's stdlib http.server with state.json polling. main.py writes state.json atomically after each step (write to a temp file then os.replace()). The server serves the bridge directory and the frontend HTML polls /api/state every 1.5 seconds via fetch(). This approach has

zero additional dependencies, runs reliably under sudo inside OrbStack, and OrbStack's automatic port bridging means the dashboard is immediately accessible on the macOS browser at localhost:8080 with no configuration.

13.6 Compromise Representation - File Markers vs On-Memory State

CybORG tracks compromise purely in simulation state objects and writes no files to disk. In the bridge, an equivalent mechanism was needed that persists across steps and is detectable via Docker exec without modifying the containers' base images.

Two approaches were considered: (1) maintain an in-memory Python dict in main.py tracking which containers are compromised; (2) write sentinel files inside the containers (/tmp/.compromised for user access, /root/.compromised for root access). In-memory state is simpler but requires all components (red agent, intrusion detector, blue agent) to share the same Python process. Sentinel files are more realistic since they represent actual artefacts an attacker would leave, and they can be detected by any component independently via Docker exec.

Sentinel files were chosen. The initial design assumed /tmp would behave as a tmpfs mount and clear automatically on container restart, providing emergent correct Restore behaviour. This assumption turned out to be wrong. In Alpine Linux, /tmp persists across Docker restart as part of the container's writable overlay layer, requiring an explicit cleanup in _restore(). This is documented as P3.11 in the Problems section. /root/.compromised was initially left intact after Restore on the assumption root compromise should persist, which also proved incorrect and is documented as P3.17.

13.7 Which CybORG Agents to Use for Evaluation

The evaluation phase required choosing which red agent types to test the blue agent against. CybORG provides several: SleepAgent, B_lineAgent, FiniteStateRedAgent, RedMeanderAgent, and others.

SleepAgent was chosen as the do-nothing baseline to measure the minimum possible penalty (only defensive action costs, no attack pressure). FiniteStateRedAgent was chosen as the primary evaluation opponent because it follows a realistic structured attack sequence (discover -> exploit -> escalate), is deterministic, and is most representative of real threat actors following defined playbooks. RedMeanderAgent was chosen as the unpredictability stress test. B_lineAgent was considered but not included as it is essentially a faster version of FiniteState and would not add meaningful new information to the evaluation.

13.8 Inference-Time Action Masking - Deployment vs Retraining

When evaluation revealed the agent was frequently selecting edge actions (indices 64-79) that are genuinely unsupported in the Containerlab environment, a decision was required: retrain the agent from scratch on a masked action space or apply masking at inference time while keeping the pre-trained weights intact.

Training-time masking would be theoretically cleaner since the policy would be shaped around the constrained action space from the start. However, retraining wasn’t practical within the

project timeline given computational constraints (Mac CPU only, no GPU access), and retraining from scratch would lose the competition-quality policy trained for hundreds of thousands of timesteps.

Inference-time masking was chosen. The implementation bypasses the agent's get_action() method and calls the actor network directly to intercept the Categorical probability distribution before any sampling. Edge action probabilities (indices 64-79) are zeroed, and the remaining distribution is renormalised. This follows standard deployment practice described by Huang & Ontañón (2022) [12], who prove that masking and sampling from the remaining distribution yields valid policy gradient updates, and Wilson et al. (2024) [13] who apply the same technique specifically in an OT cyber security RL context. The masking is opt-in via a mask_edge_actions=True flag and the CSV output is suffixed _masked vs _baseline so results from both conditions are clearly distinguishable.

The main trade-off acknowledged is that the masked agent isn't the original trained agent but rather the same agent with an inference-time constraint applied. The relative probabilities among the remaining 65 actions may not be ideal since the agent was not optimised for this constrained space. However, this is framed as a sim-to-real deployment constraint rather than a policy improvement, consistent with how real deployed systems handle unsupported operations: preventing the system from selecting actions that would fail at execution time rather than retraining the policy around environment limitations.

14 Ethics

This project involves autonomous agents performing actions on network infrastructure, which raises several ethical and safety considerations.

All testing was conducted exclusively on isolated, self-controlled networks, either the CybORG simulation environment or a locally deployed Containerlab network running on a personal laptop. No testing was performed on public networks, institutional networks, or ATU infrastructure. Running autonomous attack-defence systems on any public or institutional network without explicit authorisation is illegal under computer misuse legislation, and this was explicitly avoided throughout the project. The red agent component simulates real attack techniques including service exploitation, privilege escalation, and disk exhaustion, all of which were executed only within the isolated Containerlab environment with no external network access.

The use of AI for autonomous cyber defence raises questions about accountability and oversight. An autonomous agent that can remove processes, restart containers, or block network traffic makes consequential decisions without human approval. In a real deployment context, appropriate safeguards would be essential: human-in-the-loop confirmation for high-impact actions (particularly Restore, which restarts a host and may cause service interruption), comprehensive audit logging of all agent decisions, rate limiting on remediation actions, and a manual override mechanism. These considerations were reflected on throughout the project.

The project also made use of AI coding assistants (Claude) for debugging support, code generation, and documentation. This usage was disclosed to the project supervisor and

acknowledged in this report. All AI-generated code was reviewed, understood, tested, and validated by the author before inclusion in the project.

15 Conclusion

This project managed to build and evaluate a complete autonomous network defence system. It demonstrated that a GNN-PPO policy trained entirely in the CybORG CAGE 4 simulation [1] can be deployed against a real containerised network [9] without retraining.
Phase 1 established a solid RL foundation through systematic benchmarking of PPO, DQN, and A2C across two Gymnasium environments. All three converged without issues. PPO was chosen for Phase 2 based on its stability and scalability to multi-agent settings.

Phase 2 built the full bridge system across six components (network monitor, intrusion detector, graph builder, agent adapter, action executor, and red agent). Each component was implemented as a Python module operating on live Docker containers managed by Containerlab. A live D3.js dashboard provides real-time visualisation of each episode.

Evaluation confirmed the central research question: the policy transfer does hold. In the unmasked bridge deployment, edge actions accounted for 69% of all selections compared to 51.5% in simulation, which was the primary action feasibility gap. After inference-time masking [12], Monitor transferred consistently (41% bridge vs 34% simulation), confirming the agent's fundamental watch-then-act strategy is preserved. The elevated Analyse rate in the bridge (39% vs 3%) is a direct and explainable consequence of observation sparsity. The agent probed more since its feature vector is less informative than in training. The sim-to-real gaps were identified and described, unsupported firewall actions and feature vector sparsity. The gaps were addressed through inference-time action masking, adding a concrete deployment case study to the sim-to-real transfer literature [11].

The system was delivered across 15 sprints with a GitHub Actions CI/CD pipeline maintained throughout, achieving 73% test coverage across 20 passing tests.

Several directions exist for extending this work. The most direct continuation is retraining the agent with action masking applied at training time rather than inference time. The current policy assigns probability mass to 16 edge actions that are never executable in the bridge, and retraining with those slots removed from the start would produce a policy optimised for the actual deployment action space. A second direction is online fine-tuning on bridge episodes. The agent is currently frozen at deployment, and allowing gradient updates from real network interactions would close the observation sparsity gap empirically rather than through manual feature engineering. Third, replacing the fixed FSM red agent with a learned adversary trained via self-play would expose blue policy weaknesses that the finite state machine never tests. So it would be like a red-blue co-evolution to produce a significantly harder and more realistic threat model. Fourth, integrating a real intrusion detection system such as Suricata or Zeek to populate the observation graph would replace the sentinel file proxies used in this work with genuine network-level signals, addressing the observation gap at source. It would also fully close the sim-to-real

gap as there would be genuine network traffic with hosts communicating rather than using sentinel files. Fifth, extracting attention weights from the GNN layers at inference time would reveal which nodes and edges the agent attends to when selecting actions. This would be distinguishing whether the policy has learned topology-aware defensive reasoning or is reacting purely to local compromise flags. Finally, training across dynamically generated topologies with varying subnet counts, host counts, and connectivity patterns would test whether the GNN has learned topology-invariant defensive behaviour or has overfit to the fixed CAGE4 layout, which is a prerequisite for deployment on real enterprise networks.

16 Handover

To whoever takes up this project, hello! “I'm limited by the technology of my time, but one day you'll figure this out.”
The full source code and project information is available Here: Network Defender Repo
The system runs inside an OrbStack Linux VM on macOS, however it is possible to set up on Windows (Follow setup instructions on GitHub repository readme). Prerequisites are OrbStack with Docker, Containerlab, and a Python 3.11 virtual environment created natively inside the Linux VM at ~/fyp-venv-linux (PyTorch .so files read from the macOS-mounted filesystem via OrbStack are prohibitively slow — the venv must live on the Linux filesystem).

To deploy the network topology:

```
cd ~/Desktop/Network_Defender_FYP/containerlab-networks
sudo containerlab deploy -t cage4-topology.yaml
```

To run a bridge episode:

```
cd ~/Desktop/Network_Defender_FYP/bridge
rm -rf __pycache__
sudo ~/fyp-venv-linux/bin/python main.py
```

The live dashboard is served automatically at http://localhost:8080 once main.py is ran. OrbStack port-forwards from the Linux VM so the dashboard opens in a standard browser with no additional configuration.

16.1 Key Files

File	Purpose
bridge/main.py	Episode loop entry point — wires all components together
bridge/network_monitor.py	Polls Docker containers to build network state
bridge/graph_builder.py	Converts network state to a 192-dim PyTorch Geometric graph
bridge/agent_adapter.py	Loads trained GNN-PPO weights and produces action integers
bridge/action_executor.py	Executes blue agent actions on live containers via Docker SDK

bridge/red_agent.py	FSM attacker — 9 actions mapped to Docker exec_run calls
bridge/intrusion_detector.py	Detects compromise via sentinel files on containers
bridge/server.py	Serves live dashboard and /api/state endpoint
bridge/evaluation.py	Automated 100-step run with CSV logging
containerlab-networks/cage4-topology.yaml	Full 25-container CAGE4 topology
frontend/index.html	D3.js dashboard — polls /api/state every 1.5 seconds

16.2 CI/CD

The GitHub Actions pipeline (.github/workflows/ci.yml) triggers on pushes to develop, feature/**, and SCRUM-**-feature/** branches and on pull requests to develop and main. It spins up a Docker Compose test topology, runs pytest with coverage, and tears it down. The final CI run shows 20 passed, 1 skipped, 73% coverage. The skipped test (test_deploy_decoy) requires a live Containerlab installation not available on GitHub hosted runners and was validated manually against the OrbStack deployment.

16.3 Known Limitations

- **Edge actions (64–79):** Implemented via ip route blackhole but have no defensive effect because the red agent operates through the Docker API and bypasses network routing entirely. Inference-time action masking is applied in agent_adapter.py to prevent the agent wasting steps on these actions.
- **Red agent start state:** The bridge red agent starts knowing all hosts. CybORG's FiniteStateRedAgent starts knowing only the contractor subnet — a known simplification documented in the report.
- **Observation sparsity:** Simulation-only metadata (OS versions, file system, service pack) is left as zero in the 192-dim feature vector. bridge/graph_builder.py encode_host() is where richer features would be added.

16.4 Known Gotchas

- All bridge scripts must run under sudo — Containerlab requires root access to the Docker network
- Always invoke Python as sudo ~/fyp-venv-linux/bin/python — the venv is not in a standard location but on the linux file system
- Always run rm -rf __pycache__ before starting any file as stale .pyc files on the OrbStack macOS mount cause silent import errors and run stale code
- If the system crashes mid-episode, stale decoy containers persist — check docker ps and redeploy the topology with sudo containerlab deploy -t cage4-topology.yaml --reconfigure before restarting (or just destroy and deploy again)

16.5 Future Development

Several directions exist for extending this work. The most direct continuation is retraining the agent with action masking applied at training time rather than inference time. The current policy

assigns probability mass to 16 edge actions that are never executable in the bridge, and retraining with those slots removed from the start would produce a policy optimised for the actual deployment action space. A second direction is online fine-tuning on bridge episodes. The agent is currently frozen at deployment, and allowing gradient updates from real network interactions would close the observation sparsity gap empirically rather than through manual feature engineering. Third, replacing the fixed FSM red agent with a learned adversary trained via self-play would expose blue policy weaknesses that the finite state machine never tests. So it would be like a red-blue co-evolution to produce a significantly harder and more realistic threat model. Fourth, integrating a real intrusion detection system such as Suricata or Zeek to populate the observation graph would replace the sentinel file proxies used in this work with genuine network-level signals, addressing the observation gap at source. It would also fully close the sim-to-real gap as there would be genuine network traffic with hosts communicating rather than using sentinel files. Fifth, extracting attention weights from the GNN layers at inference time would reveal which nodes and edges the agent attends to when selecting actions. This would be distinguishing whether the policy has learned topology-aware defensive reasoning or is reacting purely to local compromise flags. Finally, training across dynamically generated topologies with varying subnet counts, host counts, and connectivity patterns would test whether the GNN has learned topology-invariant defensive behaviour or has overfit to the fixed CAGE4 layout, which is a prerequisite for deployment on real enterprise networks.

17 Appendix

17.1 Repository: Phase 1

GitHUB: https://github.com/DavidJ7705/Learning_RL_FYP

17.2 Repository: Phase 2

GitHub: https://github.com/DavidJ7705/Network_Defender_FYP

17.3 Cage4-topology.yaml

```
# CAGE4 Defense Network Topology
#
# Deploy:
#   cd ~/Desktop/Network_Defender_FYP/Containerlab-networks
#   sudo Containerlab deploy -t cage4-topology.yaml
#
# Destroy:
#   sudo Containerlab destroy -t cage4-topology.yaml
#
# Note: topology must be deployed before running any file in bridge/

name: cage4-defense-network

topology:
  nodes:

    # ************************************************************************
    # INTERNET ROUTER
    # ************************************************************************
    internet-router:
      kind: linux
      image: alpine:latest
      exec:
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 10.0.0.1/30 dev eth1 # Link to Restricted Zone A Router
        - ip addr add 10.0.0.5/30 dev eth2 # Link to Restricted Zone B Router
        - ip addr add 10.0.0.9/30 dev eth3 # Link to Contractor Network Router
        - ip addr add 10.0.0.13/30 dev eth4 # Link to Public Access Zone Router
```

```
    #Return routes - Zone A
    - ip route add 10.0.1.0/24 via 10.0.0.2
    - ip route add 10.0.2.0/24 via 10.0.0.2

    #Return routes - Zone B
    - ip route add 10.0.3.0/24 via 10.0.0.6
    - ip route add 10.0.4.0/24 via 10.0.0.6

    #Return routes - Contractor Network
    - ip route add 10.0.5.0/24 via 10.0.0.10
    #Return routes - Public Access Zone
    - ip route add 10.0.6.0/24 via 10.0.0.14
    - ip route add 10.0.7.0/24 via 10.0.0.14
    - ip route add 10.0.8.0/24 via 10.0.0.14

# ************************************************************************
# RESTRICTED ZONE A
# ************************************************************************
restricted-zone-a-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.0.2/30 dev eth1
    - ip addr add 10.0.1.1/30 dev eth2
    - ip addr add 10.0.1.5/30 dev eth3
    - ip addr add 10.0.1.9/30 dev eth4
    - ip addr add 10.0.1.13/30 dev eth5
    - ip route replace default via 10.0.0.1
    - ip route add 10.0.2.0/24 via 10.0.1.2

restricted-zone-a-server-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.1.6/30 dev eth1
```

```
    - ip route replace default via 10.0.1.5

restricted-zone-a-server-1:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.1.10/30 dev eth1
    - ip route replace default via 10.0.1.9

restricted-zone-a-user-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.1.14/30 dev eth1
    - ip route replace default via 10.0.1.13

# ************************************************************************
# OPERATIONAL ZONE A
# ************************************************************************
operational-zone-a-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.1.2/30 dev eth1
    - ip addr add 10.0.2.1/30 dev eth2
    - ip addr add 10.0.2.5/30 dev eth3
    - ip route replace default via 10.0.1.1

operational-zone-a-server-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.2.2/30 dev eth1
    - ip route replace default via 10.0.2.1

operational-zone-a-user-0:
```

```
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.2.6/30 dev eth1
    - ip route replace default via 10.0.2.5

# ************************************************************************
# RESTRICTED ZONE B
# ************************************************************************
restricted-zone-b-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.0.6/30 dev eth1
    - ip addr add 10.0.3.1/30 dev eth2
    - ip addr add 10.0.3.5/30 dev eth3
    - ip addr add 10.0.3.9/30 dev eth4
    - ip route replace default via 10.0.0.5
    - ip route add 10.0.4.0/24 via 10.0.3.2

restricted-zone-b-server-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.3.6/30 dev eth1
    - ip route replace default via 10.0.3.5

restricted-zone-b-user-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.3.10/30 dev eth1
    - ip route replace default via 10.0.3.9

# ************************************************************************
# OPERATIONAL ZONE B
```

```
# ***********************************************************************
operational-zone-b-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.3.2/30 dev eth1
    - ip addr add 10.0.4.1/30 dev eth2
    - ip addr add 10.0.4.5/30 dev eth3
    - ip route replace default via 10.0.3.1

operational-zone-b-server-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.4.2/30 dev eth1
    - ip route replace default via 10.0.4.1

operational-zone-b-user-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.4.6/30 dev eth1
    - ip route replace default via 10.0.4.5

# ***********************************************************************
# CONTRACTOR NETWORK
# ***********************************************************************
contractor-network-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.0.10/30 dev eth1
    - ip addr add 10.0.5.1/30 dev eth2
    - ip addr add 10.0.5.5/30 dev eth3
    - ip addr add 10.0.5.9/30 dev eth4
```

```
    - ip route replace default via 10.0.0.9

contractor-network-server-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.5.2/30 dev eth1
    - ip route replace default via 10.0.5.1

contractor-network-user-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.5.6/30 dev eth1
    - ip route replace default via 10.0.5.5

contractor-network-user-1:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.5.10/30 dev eth1
    - ip route replace default via 10.0.5.9


# ***********************************************************************
# PUBLIC ACCESS ZONE
# ***********************************************************************
public-access-zone-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.0.14/30 dev eth1
    - ip addr add 10.0.6.1/30 dev eth2
    - ip addr add 10.0.6.5/30 dev eth3
    - ip addr add 10.0.6.9/30 dev eth4
    - ip route replace default via 10.0.0.13
```

```
    - ip route add 10.0.7.0/24 via 10.0.6.2
    - ip route add 10.0.8.0/24 via 10.0.6.6

public-access-zone-user-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.6.10/30 dev eth1
    - ip route replace default via 10.0.6.9

# ************************************************************************
# ADMIN NETWORK
# ************************************************************************
admin-network-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
    - ip addr add 10.0.6.2/30 dev eth1
    - ip addr add 10.0.7.1/30 dev eth2
    - ip route replace default via 10.0.6.1

admin-network-user-0:
  kind: linux
  image: alpine:latest
  exec:
    - ip addr add 10.0.7.2/30 dev eth1
    - ip route replace default via 10.0.7.1

# ************************************************************************
# OFFICE NETWORK
# ************************************************************************
office-network-router:
  kind: linux
  image: alpine:latest
  exec:
    - sysctl -w net.ipv4.ip_forward=1
```

```
    - ip addr add 10.0.6.6/30 dev eth1
    - ip addr add 10.0.8.1/30 dev eth2
    - ip addr add 10.0.8.5/30 dev eth3
    - ip route replace default via 10.0.6.5

  office-network-user-0:
    kind: linux
    image: alpine:latest
    exec:
      - ip addr add 10.0.8.2/30 dev eth1
      - ip route replace default via 10.0.8.1

  office-network-user-1:
    kind: linux
    image: alpine:latest
    exec:
      - ip addr add 10.0.8.6/30 dev eth1
      - ip route replace default via 10.0.8.5

# ************************************************************************
# LINKS
# ************************************************************************
links:
  # Internet Router upstream links
  - endpoints: ["internet-router:eth1", "restricted-zone-a-router:eth1"]
  - endpoints: ["internet-router:eth2", "restricted-zone-b-router:eth1"]
  - endpoints: ["internet-router:eth3", "contractor-network-router:eth1"]
  - endpoints: ["internet-router:eth4", "public-access-zone-router:eth1"]

  # Restricted Zone A links
  - endpoints: ["restricted-zone-a-router:eth2", "operational-zone-a-router:eth1"]
  - endpoints: ["restricted-zone-a-router:eth3", "restricted-zone-a-server-0:eth1"]
  - endpoints: ["restricted-zone-a-router:eth4", "restricted-zone-a-server-1:eth1"]
  - endpoints: ["restricted-zone-a-router:eth5", "restricted-zone-a-user-0:eth1"]


  # Operational Zone A links
```

```
- endpoints: ["operational-zone-a-router:eth2", "operational-zone-a-server-0:eth1"]
- endpoints: ["operational-zone-a-router:eth3", "operational-zone-a-user-0:eth1"]

# Restricted Zone B links
- endpoints: ["restricted-zone-b-router:eth2", "operational-zone-b-router:eth1"]
- endpoints: ["restricted-zone-b-router:eth3", "restricted-zone-b-server-0:eth1"]
- endpoints: ["restricted-zone-b-router:eth4", "restricted-zone-b-user-0:eth1"]

# Operational Zone B links
- endpoints: ["operational-zone-b-router:eth2", "operational-zone-b-server-0:eth1"]
- endpoints: ["operational-zone-b-router:eth3", "operational-zone-b-user-0:eth1"]

# Contractor Network links
- endpoints: ["contractor-network-router:eth2", "contractor-network-server-0:eth1"]
- endpoints: ["contractor-network-router:eth3", "contractor-network-user-0:eth1"]
- endpoints: ["contractor-network-router:eth4", "contractor-network-user-1:eth1"]

# Public Access Zone links
- endpoints: ["public-access-zone-router:eth2", "admin-network-router:eth1"]
- endpoints: ["public-access-zone-router:eth3", "office-network-router:eth1"]
- endpoints: ["public-access-zone-router:eth4", "public-access-zone-user-0:eth1"]

# Admin Network links
- endpoints: ["admin-network-router:eth2", "admin-network-user-0:eth1"]

# Office Network links
- endpoints: ["office-network-router:eth2", "office-network-user-0:eth1"]
- endpoints: ["office-network-router:eth3", "office-network-user-1:eth1"]
```

17.4 Inspect_weights.py (view 192 input dimension)

```
import torch
import os
import sys
from wrapper.observation_graph import ObservationGraph
```

```
WEIGHTS_PATH = os.path.join(
    os.path.dirname(__file__), "..", "trained-agent", "weights", "gnn_ppo-0.pt"
)

data = torch.load(WEIGHTS_PATH, map_location="cpu", weights_only=False)
args, kwargs = data["agent"]

model_input_dimension = args[0]

print(f"Agent constructor args: {args}")
print(f"Agent constructor kwargs: {list(kwargs.keys())}")
print(f"\ninput_dimension = {model_input_dimension}")

#seeing where the 192 comes from
graph_dim = ObservationGraph.DIM
print(f"ObservationGraph.DIM = {graph_dim}")


difference = model_input_dimension - graph_dim
print(f"Extra features added: {difference}")
print("+5 comes from:\n +2 -> Tabular host feature (compromised, scanned)\n +3 -> Message features (2 message bits
+ is_received flag)")

# for i, label in enumerate(ObservationGraph.FEAT_MAP):
#     print(f"[{i}]{label}")

from wrapper.globals import ROUTERS

WRAPPER_LABELS = [
    "tabular: was_compromised",
    "tabular: was_scanned",
    "message: bit_0",
    "message: bit_1",
    "message: is_received",
]

all_labels = (
```

```
    list(ObservationGraph.FEAT_MAP) +   # 0-177
    ROUTERS +                           # 178-186  (subnet one-hot encoding)
    WRAPPER_LABELS                      # 187-191  (graph_wrapper additions)
)

print(f"\nFull feature vector  (0 - {model_input_dimension - 1})\n")
for i, label in enumerate(all_labels):
    print(f"[{i}]  {label}")
```

17.5 Inspect_weights.py - Raw Output (192 feature dimension)

```
[0]  <class 'wrapper.nodes.SystemNode'>
[1]  <class 'wrapper.nodes.ConnectionNode'>
[2]  <class 'wrapper.nodes.FileNode'>
[3]  <class 'wrapper.nodes.InternetNode'>
[4]  ('x86', <Architecture.x86: 1>)
[5]  ('x64', <Architecture.x64: 2>)
[6]  ('UNKNOWN', <Architecture.UNKNOWN: 3>)
[7]  ('UNKNOWN', <OperatingSystemDistribution.UNKNOWN: 1>)
[8]  ('WINDOWS_XP', <OperatingSystemDistribution.WINDOWS_XP: 2>)
[9]  ('WINDOWS_SVR_2003', <OperatingSystemDistribution.WINDOWS_SVR_2003: 3>)
[10]  ('WINDOWS_SVR_2008', <OperatingSystemDistribution.WINDOWS_SVR_2008: 4>)
[11]  ('WINDOWS_7', <OperatingSystemDistribution.WINDOWS_7: 5>)
[12]  ('WINDOWS_8', <OperatingSystemDistribution.WINDOWS_8: 6>)
[13]  ('WINDOWS_10', <OperatingSystemDistribution.WINDOWS_10: 7>)
[14]  ('UBUNTU', <OperatingSystemDistribution.UBUNTU: 8>)
[15]  ('KALI', <OperatingSystemDistribution.KALI: 9>)
[16]  ('WINDOWS_SVR_2003SP2', <OperatingSystemDistribution.WINDOWS_SVR_2003SP2: 10>)
[17]  ('WINDOWS_VISTA', <OperatingSystemDistribution.WINDOWS_VISTA: 11>)
[18]  ('WINDOWS_SVR_2008SP1', <OperatingSystemDistribution.WINDOWS_SVR_2008SP1: 12>)
[19]  ('WINDOWS_SVR_2008R2', <OperatingSystemDistribution.WINDOWS_SVR_2008R2: 13>)
[20]  ('WINDOWS_7SP1', <OperatingSystemDistribution.WINDOWS_7SP1: 14>)
[21]  ('DRONE_LINUX', <OperatingSystemDistribution.DRONE_LINUX: 15>)
[22]  ('UNKNOWN', <OperatingSystemType.UNKNOWN: 1>)
[23]  ('WINDOWS', <OperatingSystemType.WINDOWS: 2>)
[24]  ('LINUX', <OperatingSystemType.LINUX: 3>)
[25]  ('UNKNOWN', <OperatingSystemVersion.UNKNOWN: 1>)
[26]  ('SP0', <OperatingSystemVersion.SP0: 2>)
[27]  ('SP1', <OperatingSystemVersion.SP1: 3>)
[28]  ('SP2', <OperatingSystemVersion.SP2: 4>)
[29]  ('SP3', <OperatingSystemVersion.SP3: 5>)
[30]  ('U18_04_3', <OperatingSystemVersion.U18_04_3: 6>)
[31]  ('U18_04', <OperatingSystemVersion.U18_04: 7>)
[32]  ('U8_04', <OperatingSystemVersion.U8_04: 8>)
[33]  ('K2019_1', <OperatingSystemVersion.K2019_1: 9>)
[34]  ('K2019_2', <OperatingSystemVersion.K2019_2: 10>)
```

```
[35] ('K2019_4', <OperatingSystemVersion.K2019_4: 11>)
[36] ('W6_2_9200_16384', <OperatingSystemVersion.W6_2_9200_16384: 12>)
[37] ('W6_1_7601', <OperatingSystemVersion.W6_1_7601: 13>)
[38] ('NT6_1', <OperatingSystemVersion.NT6_1: 14>)
[39] ('UNKNOWN', <OperatingSystemKernelVersion.UNKNOWN: 1>)
[40] ('L2_6_24', <OperatingSystemKernelVersion.L2_6_24: 2>)
[41] ('L4_15_0_1057_AWS', <OperatingSystemKernelVersion.L4_15_0_1057_AWS: 3>)
[42] ('L5_2_0', <OperatingSystemKernelVersion.L5_2_0: 4>)
[43] ('L5_3_0', <OperatingSystemKernelVersion.L5_3_0: 5>)
[44] ('UNKNOWN', <OperatingSystemPatch.UNKNOWN: 1>)
[45] ('FILE_1', <OperatingSystemPatch.FILE_1: 2>)
[46] ('Q147222', <OperatingSystemPatch.Q147222: 3>)
[47] ('KB911164', <OperatingSystemPatch.KB911164: 4>)
[48] ('MS17_010', <OperatingSystemPatch.MS17_010: 5>)
[49] ('KB4500331', <OperatingSystemPatch.KB4500331: 6>)
[50] ('KB4499149', <OperatingSystemPatch.KB4499149: 7>)
[51] ('KB4499180', <OperatingSystemPatch.KB4499180: 8>)
[52] ('KB4499164', <OperatingSystemPatch.KB4499164: 9>)
[53] ('KB4499175', <OperatingSystemPatch.KB4499175: 10>)
[54] crown_jewel
[55] user
[56] server
[57] router
[58] ('UNKNOWN', <ProcessName.UNKNOWN: 1>)
[59] ('SVCHOST', <ProcessName.SVCHOST: 2>)
[60] ('INIT', <ProcessName.INIT: 3>)
[61] ('CRON', <ProcessName.CRON: 4>)
[62] ('UDEVD', <ProcessName.UDEVD: 5>)
[63] ('JSVC', <ProcessName.JSVC: 6>)
[64] ('SSHD', <ProcessName.SSHD: 7>)
[65] ('MYSQLD_SAFE', <ProcessName.MYSQLD_SAFE: 8>)
[66] ('MYSQLD', <ProcessName.MYSQLD: 9>)
[67] ('SMBD', <ProcessName.SMBD: 10>)
[68] ('SMTP', <ProcessName.SMTP: 11>)
[69] ('FEMITTER', <ProcessName.FEMITTER: 12>)
[70] ('APACHE2', <ProcessName.APACHE2: 13>)
[71] ('EXPLORER', <ProcessName.EXPLORER: 14>)
[72] ('LSASS', <ProcessName.LSASS: 15>)
[73] ('WINLOGON', <ProcessName.WINLOGON: 16>)
[74] ('SMSS', <ProcessName.SMSS: 17>)
[75] ('SYSTEM', <ProcessName.SYSTEM: 18>)
[76] ('SYSTEM_IDLE_PROCESS', <ProcessName.SYSTEM_IDLE_PROCESS: 19>)
[77] ('SERVICES', <ProcessName.SERVICES: 20>)
[78] ('SHELL', <ProcessName.SHELL: 21>)
[79] ('TELNET', <ProcessName.TELNET: 22>)
[80] ('SLEEP', <ProcessName.SLEEP: 23>)
[81] ('JAVA', <ProcessName.JAVA: 24>)
[82] ('PS', <ProcessName.PS: 25>)
```

```
[83]  ('VELOCLIENT', <ProcessName.VELOCLIENT: 26>)
[84]  ('POWERSHELL', <ProcessName.POWERSHELL: 27>)
[85]  ('CMD', <ProcessName.CMD: 28>)
[86]  ('OTSERVICE', <ProcessName.OTSERVICE: 29>)
[87]  ('UNKNOWN', <ProcessType.UNKNOWN: 1>)
[88]  ('SSH', <ProcessType.SSH: 2>)
[89]  ('SVCHOST', <ProcessType.SVCHOST: 3>)
[90]  ('SMB', <ProcessType.SMB: 4>)
[91]  ('SMTP', <ProcessType.SMTP: 5>)
[92]  ('FEMITTER', <ProcessType.FEMITTER: 6>)
[93]  ('WEBSERVER', <ProcessType.WEBSERVER: 7>)
[94]  ('NETCAT', <ProcessType.NETCAT: 8>)
[95]  ('RDP', <ProcessType.RDP: 9>)
[96]  ('REVERSE_SESSION_HANDLER', <ProcessType.REVERSE_SESSION_HANDLER: 10>)
[97]  ('REVERSE_SESSION', <ProcessType.REVERSE_SESSION: 11>)
[98]  ('MYSQL', <ProcessType.MYSQL: 12>)
[99]  suspicious pid
[100]  is decoy
[101]  is default
[102]  is_ephemeral
[103]  ('UNKNOWN', <FileType.UNKNOWN: 1>)
[104]  ('SVCHOST', <FileType.SVCHOST: 2>)
[105]  ('PASSWD', <FileType.PASSWD: 3>)
[106]  ('SHADOW', <FileType.SHADOW: 4>)
[107]  ('FLAG', <FileType.FLAG: 5>)
[108]  ('SMBCLIENT', <FileType.SMBCLIENT: 6>)
[109]  ('NMAP', <FileType.NMAP: 7>)
[110]  ('DirtyCowCode', <FileType.DirtyCowCode: 8>)
[111]  ('DirtyCowExe', <FileType.DirtyCowExe: 9>)
[112]  ('PYTHON', <FileType.PYTHON: 10>)
[113]  ('GCC', <FileType.GCC: 11>)
[114]  ('UDEV141CODE', <FileType.UDEV141CODE: 12>)
[115]  ('UDEV141EXE', <FileType.UDEV141EXE: 13>)
[116]  ('NC_REVERSE_SHELL', <FileType.NC_REVERSE_SHELL: 14>)
[117]  ('NC', <FileType.NC: 15>)
[118]  ('UNKNOWN', <Path.UNKNOWN: 1>)
[119]  ('WINDOWS', <Path.WINDOWS: 2>)
[120]  ('WINDOWS_SYSTEM', <Path.WINDOWS_SYSTEM: 3>)
[121]  ('SYSTEM', <Path.SYSTEM: 4>)
[122]  ('TEMP', <Path.TEMP: 5>)
[123]  ('SBIN', <Path.SBIN: 6>)
[124]  ('BIN', <Path.BIN: 7>)
[125]  ('USR_SBIN', <Path.USR_SBIN: 8>)
[126]  ('USR_BIN', <Path.USR_BIN: 9>)
[127]  ('ETC', <Path.ETC: 10>)
[128]  ('ADMINISTRATOR_DESKTOP', <Path.ADMINISTRATOR_DESKTOP: 11>)
[129]  ('WEBSERVER', <Path.WEBSERVER: 12>)
[130]  ('EXPLOIT', <Path.EXPLOIT: 13>)
```

```
[131] permissions
[132] permissions
[133] permissions
[134] permissions
[135] permissions
[136] permissions
[137] permissions
[138] permissions
[139] permissions
[140] permissions
[141] permissions
[142] permissions
[143] permissions
[144] permissions
[145] permissions
[146] permissions
[147] permissions
[148] permissions
[149] permissions
[150] permissions
[151] permissions
[152] permissions
[153] permissions
[154] permissions
[155] ('UNKNOWN', <FileVersion.UNKNOWN: 1>)
[156] ('U4_2_4_1', <FileVersion.U4_2_4_1: 2>)
[157] ('D9_2_1_21', <FileVersion.D9_2_1_21: 3>)
[158] ('OPENBSD', <FileVersion.OPENBSD: 4>)
[159] ('UNKNOWN', <FileType.UNKNOWN: 1>)
[160] ('SVCHOST', <FileType.SVCHOST: 2>)
[161] ('PASSWD', <FileType.PASSWD: 3>)
[162] ('SHADOW', <FileType.SHADOW: 4>)
[163] ('FLAG', <FileType.FLAG: 5>)
[164] ('SMBCLIENT', <FileType.SMBCLIENT: 6>)
[165] ('NMAP', <FileType.NMAP: 7>)
[166] ('DirtyCowCode', <FileType.DirtyCowCode: 8>)
[167] ('DirtyCowExe', <FileType.DirtyCowExe: 9>)
[168] ('PYTHON', <FileType.PYTHON: 10>)
[169] ('GCC', <FileType.GCC: 11>)
[170] ('UDEV141CODE', <FileType.UDEV141CODE: 12>)
[171] ('UDEV141EXE', <FileType.UDEV141EXE: 13>)
[172] ('NC_REVERSE_SHELL', <FileType.NC_REVERSE_SHELL: 14>)
[173] ('NC', <FileType.NC: 15>)
[174] ('UNKNOWN', <Vendor.UNKNOWN: 1>)
[175] is new
[176] density
[177] signed
[178] admin_network_subnet_router
```

[179] contractor_network_subnet_router
[180] internet_subnet_router
[181] office_network_subnet_router
[182] operational_zone_a_subnet_router
[183] operational_zone_b_subnet_router
[184] public_access_zone_subnet_router
[185] restricted_zone_a_subnet_router
[186] restricted_zone_b_subnet_router
[187] tabular: was_compromised
[188] tabular: was_scanned
[189] message: bit_0
[190] message: bit_1
[191] message: is_received

18 AI Usage

This project made use of AI coding assistants throughout development, primarily Claude (Anthropic) and ChatGPT (OpenAI), with occasional use of Amazon Q. All AI-generated output was reviewed, tested, and validated before inclusion in the project.

Claude was the primary assistant used across the project. Key uses included: generating the initial observation graph converter architecture for bridging CybORG observations to a Containerlab network; writing Docker action test scripts including the interactive container action tester and live container watcher; generating the pytest test suite for bridge components from CI-compatible Docker fixtures; building the compare.py action distribution comparison script and wiring the live D3.js frontend to the backend state server; and generating the project README. Claude was also used during report writing for structural suggestions, proofreading, and formatting assistance.

ChatGPT was used to investigate where the +5 dimension offset in the GNN-PPO input came from, correctly identifying that it originated from two tabular host features (scanned, compromised) and three message features appended after the base ObservationGraph dimension. ChatGPT was also used to help structure the CybORG results analysis and plotting script.

Amazon Q was used to extract and print the CybORG CAGE 4 topology structure from the EnterpriseScenarioGenerator, used to understand subnet layout and inter-router links when designing the Containerlab topology.

All AI assistance was limited to implementation support and debugging. The research direction, architectural decisions, evaluation methodology, and interpretation of results were all my own. AI tools had no access to the live Containerlab environment, training results, or Jira board. All generated code was understood and tested before use, and any code that didn't work as expected was debugged and corrected manually.

19 References

[1] TTCP CybORG Team, "CAGE Challenge 4," Feb. 2024. [Online]. Available: https://cage-challenge.github.io/cage-challenge-4/

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, Jul. 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[3] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529-533, Feb. 2015. [Online]. Available: https://www.nature.com/articles/nature14236

[4] V. Mnih et al., "Asynchronous Methods for Deep Reinforcement Learning," in Proc. Int. Conf. Machine Learning (ICML), 2016. [Online]. Available: https://arxiv.org/abs/1602.01783

[5] Farama Foundation, "Gymnasium Documentation," 2024. [Online]. Available: https://gymnasium.farama.org/introduction/basic_usage/

[6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," J. Machine Learning Research, vol. 22, no. 268, pp. 1-8, 2021. [Online]. Available: https://stable-baselines3.readthedocs.io/

[7] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," arXiv:1609.02907, 2016. [Online]. Available: https://arxiv.org/abs/1609.02907

[8] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019. [Online]. Available: https://arxiv.org/abs/1903.02428

[9] Containerlab, "Containerlab Documentation," 2024. [Online]. Available: https://Containerlab.dev

[10] A. Baillie et al., "CybORG++: An Enhanced Gym for the Development of Autonomous Cyber Agents," arXiv:2410.16324, Oct. 2024. [Online]. Available: https://arxiv.org/abs/2410.16324

[11] T. Zhao et al., "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey," in Proc. IEEE Symp. Series Comput. Intell. (SSCI), 2020. [Online]. Available: https://arxiv.org/abs/2009.05823

[12] S. Huang and S. Ontañón, "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," in Proc. 35th Int. FLAIRS Conf., 2022. [Online]. Available: https://arxiv.org/abs/2006.14171

[13] R. Wilson et al., "Applying Action Masking and Curriculum Learning Techniques to Proximal Policy Optimisation for OT Cyber Security Using Reinforcement Learning," in Proc. CAMLIS, 2024.

[14] G. Varricchione et al., "Pure-Past Action Masking," in Proc. AAAI Conf. Artificial Intelligence, 2024. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/30199

[15] Y. Hou et al., "Exploring Invalid Action Masking for Reinforcement Learning and Quality-Diversity in Real-Time Strategy Games," Applied Sciences, vol. 13, 2023.

[16] A. Stolz et al., "Excluding the Irrelevant: Focusing RL through Continuous Action Masking," in Proc. NeurIPS, 2024.

[17] E. Standen et al., "CybORG: A Gym for the Development of Autonomous Cyber Agents," arXiv:2108.09118, 2021. [Online]. Available: https://arxiv.org/abs/2305.17246

[18] cage-challenge/CybORG, GitHub. [Online]. Available: https://github.com/cage-challenge/CybORG

[19] Cybermonic, "CAGE 4 Submission," GitHub, 2024. [Online]. Available: https://github.com/cybermonic/cage-4-submission

[20] Docker, Inc., "docker container exec," Docker Docs. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/exec/

[21] Docker, Inc., "docker container restart," Docker Docs. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/restart/

[22] PyTorch, "torch.load," PyTorch Documentation. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.load.html

[23] PyTorch Geometric, "Introduction by Example," PyG Documentation, v2.6.1. [Online]. Available: https://pytorch-geometric.readthedocs.io/en/2.6.1/get_started/introduction.html

[24] Containerlab, "Topology Definition File," Containerlab.dev. [Online]. Available: https://Containerlab.dev/manual/topo-def-file/

[25] Containerlab, "Network," Containerlab.dev. [Online]. Available: https://Containerlab.dev/manual/network/

[26] Containerlab, "macOS," Containerlab.dev. [Online]. Available: https://Containerlab.dev/macos/